INF5130 - Algorithmique

Manuel d'exercices

Andrey Martinez Cruz

Automne 2026

Table des matières

0.1 Changement 4
0.2 Approche incrémentale 4
0.3 Annotation 4
0.4 Convention d'écriture d'algorithmes 5
0.5 Catégories d'exercices 5
0.5.1 Exercices normaux 5
0.5.2 Exercices extras 5

1 Rappel **6**
1.1 Série 1 6
1.2 Extra 9

2 Complexité asymptotique **11**
2.1 Série 2 13
2.2 Extra 14

3 Équation de récurrences **18**
3.1 Série 3 20
3.2 Extra 22

4 Diviser pour régner **24**
4.1 Série 4 25
4.2 Extra 29

5 Programmation dynamique **31**
5.1 Série 5 32
5.2 Extra 33

6 Algorithmes gloutons **35**
6.1 Série 6 36
6.2 Extra 37

7 Algorithmes sur les graphes **39**
7.1 Série 7 40
7.2 Extra 44

8 Analyse amortie **46**
8.1 Série 8 47
8.2 Extra 49

9 NP complétude **54**
9.1 Série 9 55
9.2 Extra 55

10 Annexe **57**
10.1 Composition 57
10.2 Quelques propriétés logarithmiques 58
10.3 Sommation 58
10.3.1 Propriétés des sommations 58
10.3.2 Sommation particulière 59
10.3.3 Exercices 60
10.4 Démonstration 62
10.4.1 Démonstrations générale 62
10.4.2 Démonstration par induction 63

Préface

0.1 Changement

Au lieu de mettre individuellement les énoncés et attendre que le corrigé soit disponible ou attendre les prochaines séries, le document a été conçu pour pouvoir ajouter incrémentalement les exercices des séries à venir au courant de la session et aussi de donner un aperçu plus général sur la matière qui sera potentiellement vu.

0.2 Approche incrémentale

Le document sera mis à jour incrémentalement, donc il est probable que certains exercices de certaines sections, les solutionnaires associés à ces exercices ne soient pas immédiatement disponibles ou bien des sections qui soient en cours de révision ou de rédaction.

0.3 Annotation

Afin de bien différencier les exercices qui sont plus durs que d'autres ou bien le type d'exercice correspondant, ces derniers auront un ou plusieurs des symboles suivants :

-  : exercice demandant de faire des traçes ou de développer les calculs
-  : exercice portant sur des démonstrations
-  : exercice ou chapitre en cours de révision et/ou en rédaction
-  : exercice plus difficile que les autres.

0.4 Convention d'écriture d'algorithmes

Sauf mention contraire, tous exercices concernant l'écriture d'algorithme incluant leur solutionnaire respectif, sera écrit dans une version simplifiée de **python** avec les particularités suivantes :

- Il n'y a aucune utilisation de boucle **for** et ceux peu importe la raison.
- Il n'y a aucune utilisation de la fonction de **range**.
- Le coût de l'opération suivante :
  `t = [x] * n`

  est de $1 +$ coût pour évaluer $x \times n$.

0.5 Catégories d'exercices

Les exercices seront divisés en deux types de catégories :

1. Les normaux
2. Celles en section extra

0.5.1 Exercices normaux

Ces exercices seront ceux qui seront abordés en laboratoire.

0.5.2 Exercices extras

Ces exercices ne seront pas abordés en laboratoire (sauf par demande exceptionnelle de certaines personnes), mais seront présent pour ceux ayant le besoin de pratiquer plus ou ont envie de plus tester leur connaissance. (C'est plus probable qu'il y ait des exercices plus durs dans cette section). **:-)**

Chapitre 1

Rappel

1.1 Série 1

Exercice 1

On considère le problème consistant à additionner deux nombres naturels exprimés en **base deux**, dont les nombres sont représentés dans des tableaux de n éléments. Ces tableaux sont respectivement appelés A et B. La somme des deux nombres doit être calculée et placée dans un tableau nommé C et contenant $n + 1$ éléments.

a) Écrivez une fonction qui prend en entrée les tableaux A et B et renvoie le tableau C.
b) Donnez la complexité temporelle de votre algorithme.

Exercice 2

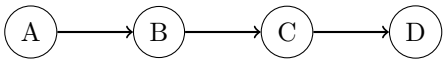
Soient A et B deux tableaux ayant la même signification que dans l'exercise précédent. On veut maintenant multiplier les nombres représentés par A et B au lieu de les additionner. Le tableau C contiendra la représentation du produit de A et B en base deux.

a) Quel doit être le nombre d'éléments du tableau C ?
b) Écrivez une fonction qui prend en entrée A et B et renvoie C.
c) Donnez la complexité temporelle de votre algorithme.

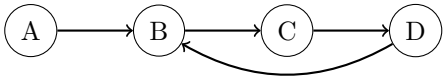
Exercice 3

Étant donnée une liste chainée L, cette dernière est cyclique si et seulement s'il existe un maillon pour lequel il existe un chemin permettant de retourner à ce dernier.

Par exemple, la liste chainée suivante ne possède pas de cycle :



En revanche, cette liste chainée en possède un :



Écrivez un algorithme qui renvoit **vrai** si la liste chainée T contient un cycle, sinon l'algorithme renvoie **faux**. Contrainte : votre algorithme doit être dans le pire des cas en $\mathcal{O}(n)$ pour la complexité temporelle et $\mathcal{O}(1)$ en complexité spatiale.

Pour cette exercise, vous avez accès seulement à ces fonctions qui sont executées en temps constant :

- OBTENIR_TETE(L) : renvoie l'emplacement du premier maillon de la liste chainée L.
- OBTENIR_SUIVANT(m) : renvoie le maillon suivant m. S'il le maillon m ne contient pas de suivant, alors la fonction renvoie la valeur *None*.

Exercice 4

Soit l'algorithme de tri nommé TRI COMPTAGE est décrivable par l'algorithme suivante :

```
1  def tri_comptage(A, k):
2      C = [0] * (k + 1)              # c1
3      B = [0] * (len(A))             # c2
4      j = 0                          # c3
5      while j < len(A):              # c4
6        C[A[j]] = C[A[j]] + 1        # c5
7        j = j + 1                    # c6
8      j = 1                          # c7
9      while j <= k:                  # c8
10         C[A[j]] = C[j] + C[j - 1] # c9
11         j = j + 1                 # c10
12     j = len(A) - 1                 # c11
13     while j >= 0:                  # c12
14       B[C[A[j]]] = A[j]            # c13
15       C[A[j]] = C[A[j]] - 1        # c14
16       j = j - 1                    # c15
17     return B                       # c16
```

Ce dernier a la particularité de donner un tableau trié sans effectuer de comparaisons. Une des contraintes de cet algorithme de tri est que le tableau qui doit être trié ne contient seulement que des nombres entiers telles que $0 \leq n \leq k$.

a) Donnez sa complexité temporelle en fonction de $c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8, c_9, c_{10}, c_{11}, c_{12}, c_{13}, c_{14}, c_{15}$ et c_{16}.

b) Donnez sa complexité spatiale.

Exercice 5

Considérez la procédure suivante :

```python
def werk(M):
    somme = 0
    i = 0
    while i < len(M):
        stack = 0
        j = i + 1
        while j < len(M):
            stack = stack + M[i][j]
            M[i][j] = 0
            j = j + 1
        rest = stack % (i + 1)
        stack = stack // (i + 1)
        p = 0
        while p <= i:
            M[i][p] = M[i][p] + stack
            if p == i:
                M[i][p] = M[i][p] + rest
            p = p + 1
        i = i + 1
```

a) Expliquez ce que fait cet algorithme.
b) Identifiez une instruction barométrique et expliquez pourquoi il s'agit d'une opération barométrique.
c) Donnez la complexité de cet algorithme en se basant sur l'opération barométrique trouvée à la question précédente.

Exercice 6

Soit le problème suivant pour lequel on souhaite obtenir la somme de tous les éléments présents sur la diagonale principale et la diagonale secondaire d'une matrice M carrée.

a) Écrivez un algorithme permettant de résoudre ce problème.
b) Selon l'algorithme écrit à la question précédente, identifiez une opération barométrique et expliquez ce choix.
c) Donnez la complexité de votre algorithme en se basant sur l'opération barométrique choisie à la question précédente.

1.2 Extra

Exercice 1

Soit un tableau T composé de nombre entiers, on voudrait savoir si ce tableau possède des doublons.

a) Écrivez un algorithme qui prend en entré un tableau d'entiers et détermine si ce tableau contient des doubles. Votre algorithme doit avoir une complexité temporelle inférieure à n^2.
b) Donnez une opération barométrique à partie de l'algortihme écrit à la question précédente.
c) Avec l'opération barométrique donnée à la question précédente, donnez la complexité de votre algorithme.

Exercice 2

Soit x, une valeur entière, et T, un tableau de nombre entiers, on voudrait savoir s'existe deux indices $i > 0, j > 0, i, j \in \mathbb{N}$ telles que

$$T[i] + T[j] = x \ (i, j \in \mathbb{N})$$

a) Écrivez un algorithme qui prend en entrée un nombre entier x et un tableau d'entiers et détermine s'il existe une paire i et j qui respecte la propriété précédemment citée.
b) Donnez une opération barométrique selon l'opération barométrique choisie à la question précédente.
c) Donnez la complexité temporelle de votre algorithme à partir de l'opération barométrique choisie à la question précédente.

Exercice 3

Considérez la fonction suivante :

```
1  def quelque_chose(n):
2      i = 0
3      while i < n:
4          j = n
5          while j >= 0:
6              k = n
7              while k >= 0:
8                  k = k // 2
9              j // 2
10         i = i + 1
```

Identifiez une opération barométrique et donnez la complexité asymptotique de cette fonction à partir de l'opération barométrique choisie.

Exercice 4

Considérez l'algorithme suivant :

```python
def algo(t):
    i = 0
    m = 0
    while i < len(t):
        j = 0
        element = t[i]
        while j <= 8:
            if j % 2 == 0:
                element = element + element ** 2
            else:
                element = element + t[i] / j
            j = j + 1
        m = m + element
        i = i + 1
    return m
```

Identifiez une opération barométrique et donnez la complexité asymptotique de cette fonction à partir de l'opération barométrique choisie.

Exercice 5

Une rotation de droite d'un tableau est un tableau dont ses éléments ont été décalés de k positions vers la droite de leur position initiale. Dans le cas des éléments se trouvant à la fin des tableaux, celle-ci sont remis au début du tableau pour pouvoir continuer à être décalé.

Par exemple, supposons qu'on a le tableau $t = [2, 4, 6, 1, 2]$ et $k = 2$, alors le tableau avec rotation à droite sera $[1, 2, 2, 4, 6]$.

Écrivez un algorithme qui prend en entrée un tableau t et qui effectue une rotation vers la droite sur ce dernier et donnez la complexité temporelle et spatiale de votre algorithme.

Chapitre 2

Complexité asymptotique

Chaque algorithme peut appartenir à une classe de croissance asymptotique tant temporelle que spatiale. Ces dernières sont déterminés selon l'ordre de croissances en fonction de la variation de la taille du problème.

Soit deux fonctions $f : \mathbb{N} \to \mathbb{R}$ et $g : \mathbb{N} \to \mathbb{R}$, alors on peut avoir les propriétés suivantes :

- $f \in \mathcal{O}(g)$ si et seulement s'il existe une paire de constantes c, n_0 tels que

$$f(n) \leq cg(n), n \geq n_0$$

- $f \in o(g)$ si et seulement s'il existe une constante $n_0 \in \mathbb{N}$ telle que pour tout $c > 0$

$$f(n) < cg(n), n \geq n_0$$

.

- $f \in \Omega(g)$ si et seulement s'il existe une paire de constantes c, n_0 tels que

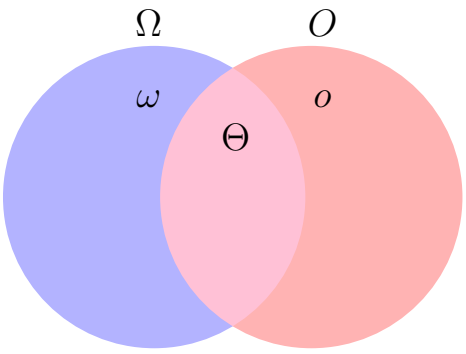
$$f(n) \geq cg(n), n \geq n_0$$

- $f \in \omega(g)$ s'il existe une constante $n_0 \in \mathbb{N}$ telle que pour tout $c > 0$

$$f(n) > cg(n), n \geq n_0$$

- $f \in \Theta(g)$ si et seulement si $f \in \mathcal{O}(g) \wedge f \in \Omega(g)$

Explication visuelle



Ensemble de fonctions ordonnées en ordre croissant selon la vitesse de croissance :

Nom de la fonction	fonction
Constante	$k \in \mathbb{N}$
Logarithmique	$\log n$
Polylogaritmique	$\log^k n, k > 0, \mathbb{N}$
Linéaire	n
Quasi-linéaire	$n \log n$
Quadratique	n^2
Polynôme	$n^k, k > 0, k \in \mathbb{N}$
Exponentielle	$k^n, k > 0, k \in \mathbb{R}$
Factorielle	$n!$
Hyper exponentielle	n^n

2.1 Série 2

Exercice 1

Soit les deux fonctions $f : \mathbb{N} \to \mathbb{N}$ et $g : \mathbb{N} \to \mathbb{N}$ telles que $f(n) \geq 0$ et $g(n) \geq 0$ pour tout n, prouvez que si $f \in \Theta(g)$, alors

$$f + g \in \Theta(\max(f, g))$$

en utilisant la définition de la notation Θ.

Exercice 2

Soit a et b, deux constantes réelles strictement positives, prouvez que

$$(n + a)^b \in \Theta(n^b)$$

pour n, un entier naturel, en utilisant la définition de la notation Θ.

Exercice 3

Soit c, une constante telle que $c \in \mathbb{R}$ et $c > -1$, prouvez que

$$\sum_{i=1}^{n} i^c \in \Theta(n^{c+1})$$

en utilisant la définition de la notation Θ.

Exercice 4

Classez les fonctions suivante par ordre croissant ; autrement dit, si la fonction $f \in o(g)$, f doit être placée à gauche de g. Si $f \in \Theta(g)$, f doit être placée au-dessus ou au-dessous de g (c'est-à-dire au même niveau que g).

$$(\frac{3}{2})^n, n^3, \log^2 n, \log n!, 2^{2^n}, (\sqrt{n})^{\log(n)}, 2^{\log(n)}, (\log n)^{\log(n)}, n!, 2^{n+1}, 2^{2n}$$

Exercice 5

Pour chacun des énoncés suivants, dites s'il est **vrai** ou **faux**. Justifiez chacune de vos réponses.

a) $2n \in \Theta(n)$
b) $(\frac{n}{\log n})^2 \in o((\frac{n}{\log n})^2)$
c) $n^{\frac{1}{2}} \in \omega(\sqrt{n})$
d) $n! \in \Theta((n+1)!)$
e) $\sum_{i=1}^{n}(2i-1) \in \Omega(n^2)$
f) $2^n \in \Omega(3^n)$
g) $\sum_{i=\lfloor \frac{n}{2} \rfloor}^{n} i \in \Theta(n)$
h) $n \log n \in \Theta(n \log_4 n)$

Exercice 6

Chacun des énoncés ci-dessous est faux. Donnez un contre-exemple pour les énoncés ci-bas.

a) Si $f \in \mathcal{O}(g)$, alors $2^f \in \mathcal{O}(2^g)$
b) $f + g \in \Theta(\min(f, g))$
c) Si $f \in \Omega(g)$, alors $g \in \Omega(f)$

2.2 Extra

Exercice 1

Démontrez que

$$\log(n!) \in \Theta(n \log n)$$

.

Exercice 2

Cet exercice consiste à comprendre la complexité asymptotique de la fonction suivante

$$f(n) = \sum_{i=1}^{n} i^d \log^k i$$

où $d > -1, k > -1$.

a) Démontrez que

$$\sum_{i=1}^{n} \log^k i \in \Theta(n \log^k n)$$

Indice : la démonstration utilisé à l'exercice précédent est utile pour démontrer cette propriété.

b) Démontrez que

$$f \in \Theta(n^{d+1} \log^k i)$$

Indice : pour démontrer cette propriété, il est recommandé d'utiliser la démonstration faite à l'exercice 1 et l'exercice 3 de la série 2.

Exercice 3

Démontrez que

$$n^2 + n + \log n \notin \Omega(n^2 \log n)$$

Exercice 4

Démontrez que

$$n \in \Omega(\log^2 n)$$

Exercice 5

Démontrez que

$$n^2 + 2n \in o(n)$$

Exercice 6

Démontrez que

$$n^3 + n \in \Theta(\sum_{i=0}^{n} i^2)$$

Exercice 7

Soit $f : \mathbb{N} \to \mathbb{R}^+$ et $g : \mathbb{N} \to \mathbb{R}^+$ tels que

$$f(n) = \log^k \log n$$
$$g(n) = \log \log^k n$$

où k, une constante, tel que $k > 0, k \in \mathbb{N}$.

Déterminez quelle est la fonction qui croit la plus vite.

Exercice 8

Démontrez que

$$n^3 + n \notin \omega(n^3)$$

Exercice 9

Démontrez que

$$3^{2n} \in \omega(3^n)$$

Exercice 10

Démontrez que

$$n! \in o(n^n)$$

sans utiliser la table de référence présent en début de chapitre.

Exercice 11

Démontrez que

$$n^2 \notin o(n \log n)$$

Exercice 12

Démontrez que

$$n \log n \notin \Theta(n \log^2 n)$$

Exercice 13

Démontrez que

$$n \log n \in \Theta(n \log_9 n)$$

Exercice 14

Soit les fonctions $h : \mathbb{N} \to \mathbb{R}^+$ et $p : \mathbb{N} \to \mathbb{R}^+$ tel que

$$h(n) = \log n!$$
$$p(n) = \log n^n$$

Déterminez s'ils grossissent asymptotiquement à la même vitesse.

Exercice 15

Soit trois fonctions $f_1 : \mathbb{N} \to \mathbb{R}^+$, $f_2 : \mathbb{N} \to \mathbb{R}^+$ et $g : \mathbb{N} \to \mathbb{R}^+$, démontrez que si $f_1 \in \Theta(g)$ et que $f_2 \in \Theta(g)$, alors

$$(f_1 + f_2) \in \Theta(g)$$

Exercice 16

Démontrez que

$$\log(x^4 + x^2) \in \Theta(\log x)$$

Exercice 17

Démontrez que

$$\left\lfloor \frac{x}{2} \right\rfloor \in \omega(\log x)$$

Exercice 18

Démontrez que

$$7n^4 + 3 \in \omega(n^3)$$

Exercice 19

Le but de cet exercice est de comprendre un peu plus sur la complexité asymptotique des fonctions se basant sur une ou plusieurs tétration(s). Une tétration, notée $k \uparrow\uparrow n$ ou ${}^n k$, est un empilement de puissance sur k n fois. Par exemple, ${}^2 3$ ou $2 \uparrow\uparrow 3$ représentent le nombre 3^{3^3}.

Soient deux fonctions $f : \mathbb{N} \to \mathbb{R}^+$ et $g : \mathbb{N} \to \mathbb{R}^+$ dont leurs définitions sont les suivabntes :

- $f(n) = k \uparrow\uparrow n$
- $g(n) = k \uparrow\uparrow (n+1)$

où $k > 1, k \in \mathbb{N}$, est-ce que les deux fonctions croîent à la même vitesse ?

Exercice 20

Démontrez que

$$n^2 \log^2(n^2 + n) \in \Theta(n^2 \log^2(n^2 + n))$$

Exercice 21

Démontrez que

$$n^3 + n \in \Theta(n^3)$$

Exercice 22

Démontrez que

$$n^2 - n \in \omega(n)$$

Exercice 23

Démontrez que

$$n - 5 \in \Theta(n)$$

Exercice 24

Démontrez que

$$n^3 + 2n^2 - n \in \Theta(n^3)$$

Exercice 25

Démontrez que

$$4n^3 - 2n^2 - n \in \mathcal{O}(n^3)$$

Chapitre 3

Équation de récurrences

Les équations de récurrences sont en coeurs des algorithmes récursifs.

Les deux types d'équations récursives qui seront travaillés dans cette section sont les équations de récurrence aux différences finies, les équations de récurrence aux divisions finies et quelques petits cas plus exotiques.

Note : il est à noter que les équations de récurrences vues dans ce cours ne représentent que des cas spécifiques et simples.

Équations de récurrence aux différences finies

Ces équations peuvent prendre la forme suivante

$$\tau(n) = \alpha\tau(n - \beta) + f(n)$$

où $\alpha > 0, \beta > 0, \alpha, \beta \in \mathbb{N}$ et $f : \mathbb{N} \to \mathbb{R}$ la fonction de recombinaison.

Équations de récurrence aux divisions finies

Ces équations peuvent prendre la forme suivante :

$$\tau(n) = \alpha\tau(\frac{n}{\beta}) + f(n)$$

où $\alpha > 0, \beta > 1, \alpha, \beta \in \mathbb{N}$ et $f : \mathbb{N} \to \mathbb{R}$ la fonction de recombinaison.

Résolution des équations de récurrences

Il existe plusieurs techniques pour évaluer une équation de récurrence.

Parmi ces techniques, en voici quelqu'uns dont certains seront abordés en cours :

- Tenter de deviner la complexité asymptotique et prouvez par induction et avec la notation Θ que l'équation de récurrence est du même ordre grandeur que l'hypothèse de départ ou dérouler la fonction pour tenter de trouver une formule permettant d'évaluer l'équation.

- Faire un arbre des appels récursifs pour trouver une formule permettant d'évaluer la complexité asymptotique de l'équation de récurrence.
- Utiliser le théorème général pour trouver la complexité asymptotique d'une équation de récurrence aux divisions finies.

Pour rappel, pour une fonction de récurrence aux divisions finies décrite à la page précédente, le théorème général (de base) se décline en un des cas suivant :

Cas	Condition	Conclusion
1	$f \in \mathcal{O}(n^{\log_\beta(\alpha)-\epsilon})$	$\tau \in \Theta(n^{\log_\beta \alpha})$
2	$f \in \Theta(n^{\log_\beta \alpha} \log^k n), k \geq 0$	$\tau \in \Theta(n^{\log_\beta \alpha} \log^k n)$
3	$f \in \Omega(n^{\log_\beta(\alpha)+\epsilon})$ et s'il existe une paire de constante $0 < k < 1$ et $n_0 \geq 0$ telles que $\alpha f(\frac{n}{\beta}) \leq k f(n)$ $(\forall n, n \geq n_0)$	$\tau \in \Theta(f)$

Note : $\epsilon > 0, \epsilon \in \mathbb{R}^+$.

Certaines équations de récurrences ne peuvent pas être résolues par la troisième méthode.

3.1 Série 3

Exercise 1

La fonction suivante prend en entrée une portion de tableau contenue entre l'indice B_I et l'indice B_S et renvoie deux nombres entiers. Les éléments du tableau sont eux-mêmes des entiers. Vous pouvez supposer que les éléments du tableau sont tous différents les uns des autres et que le nombre d'éléments de la portion de tableau sont tous différents les uns des autres et que le nombre d'éléments de la portion de tableau est pair et plus grand que 0.

```
1  def bidon(tab, bi, bs):
2      if tab[bi] >= tab[bi + 1]:
3          w = tab[bi]
4          x = tab[bi + 1]
5      else:
6          w = tab[bi + 1]
7          x = tab[bi]
8      if  bs <= (bi + 1):
9          n1 = w
10         n2 = x
11     else:
12         (y, z) = bidon(tab, bi + 2, bs)
13         if w >= y:
14             n1 = w
15         else:
16             n1 = y
17         if x <= z:
18             n2 = x
19         else:
20             n2 = z
21         return (n1, n2)
```

a) Supposons que $B_I = 0, B_S = 5$ et que Tab contient les éléments -7, 2, -3, 5, 0 , 1. Faites la trace de l'appel BIDON(TAB, B_I, B_S) : pour chaque appel à BIDON() dans la pile d'exécution, donnez les valeurs de B_I, B_S et les valeurs retournées n_1, n_2, si ces dernières ne sont pas encore connues mettez « ?».

b) Décrivez ce que fait BIDON() en général.

c) Soit $T(n)$ le nombre de comparaisons effectués par un appel de BIDON() lorsque le nombre d'éléments de la portion de tableau est égal à n. La comparaison $B_S \leq B_I + 1$ ne compte pas... Écrivez l'équation de récurrence associée à cet algorithme.

d) Résolvez l'équation de récurrence que vous avez trouvée à la question précédente. Vous pouvez la résoudre de manière exacte.

Exercise 2

Voici un algorithme qui permet de résoudre le problème des tours de Hanoi.

```
1  def deplacer(n, t1, t2, t3):
2      if n == 1:
3          print("Déplacler un disque de la tour t1 à la tour t2")
4      else:
5          deplacer(n - 1, t1, t3, t2)
6          print("Déplacer un disque de la tour t1 à la tour t2")
7          deplacer(n - 1, t3, t2, t1)
```

DEPLACER(n, T_1, T_2, T_3) a pour effet d'afficher tous les les déplacements requis pour enfiler n disques sur la tour T_2. Au départ les disques sont enfilés sur la tour T_1.

a) Soit $T(n)$, le nombre de déplacements effectués, ou ce qui revient au même, le nombre de message affichés par la procédure DEPLACER lorsqu'elle est appelée avec n comme paramètre. En supposant que l'affichage d'un message prend un temps égal à 1, écrivez l'équation de récurrence associée à $T(n)$.
b) Résolvez l'équation de récurrence trouvée à la question précédente. Vous pouvez la résoudre de manière exacte.

Exercise 3

Soit une matrice M composée d'entier, un élément est considéré comme le maximum global si et seulement s'il est le plus grand élément de la matrice.

a) Écrivez un algorithme **récursif** qui prend une matrice en entrée et renvoie le plus maximum global de cette matrice.
b) Donnez l'équation de récurrence de votre algortihme.
c) Donnez une estimation de la complexité asymptotique de votre équation de récurrence.

Exercise 4

Écrivez un algorithme efficace pour calculer la racine carrée entière d'un entier $n \geq 2$. La racine carrée entière d'un entier n supérieur ou égal à 2. La racine carrée entière de n est le plus grand entier positif r tel que $r^2 \leq n$. Par «algorithme efficace», on entend un algorithme dont le temps d'exécution est proportionnel à $\log n$.

Exercise 5

Pour chacune des équations de récurrences suivantes, trouvez une fonction f aussi simple que possible telle que $\tau \in \Theta(f)$

a) $\tau(n) = 7\tau(\frac{n}{2}) + n^3$
b) $\tau(n) = 4\tau(\frac{n}{3}) + n$
c) $\tau(n) = n\tau(n-1) + 1$
d) $\tau(n) = 8\tau(\frac{n}{2}) + n^3$
e) $\tau(n) = 2\tau(n-1) + n^2$
g) $\tau(n) = 2\tau(\frac{n}{3}) + \log n^2$
h) $\tau(n) = 4\tau(\frac{n}{3}) + n^{\log_3 4}$
i) $\tau(n) = 7\tau(\frac{n}{3}) + n^2 \log n^3$
j) $\tau(n) = 5\tau(\frac{n}{3}) + n^{\frac{3}{2}}$

Exercise 6

Le temps d'exécution de l'algorithme A est donné par l'équation de récurrence

$$\tau_A(n) = 7\tau_A(\frac{n}{2}) + n^2$$

, alors que celui de l'algorithme A' est donné par l'équation

$$\tau_{A'} = c\tau'_A(\frac{n}{4}) + n^2$$

(où c est un entier positif inconnu). Supposez que $\tau_{A'} \in o(\tau_A)$. Quelle est la plus grande valeur possible pour c ?

Exercise 7

Résolvez les équations de récurrences ci-bas **sans** utiliser le théorème général.

a) $\tau(n) = 2\tau(\frac{n}{3}) + n$
b) $\tau(n) = 4\tau(\frac{n}{2}) + \sqrt{n}$
c) $\tau(n) = 2\tau(\frac{n}{2}) + n$
d) $\tau(n) = 2\tau(\frac{n}{3}) + \log n$
e) $\tau(n) = 4\tau(\frac{n}{4}) + n \log n$

3.2 Extra

Exercice 1

Considérez l'équation de récurrence suivante :

$$\tau(n) = \tau(\sqrt{n}) + 1,$$
$$\tau(2) = 1$$

En supposant que n est de la forme 2^{2^p} pour un entier $p \in \mathbb{N}$. Donnez la complexité asymptotique de cette équation.

Exercice 2

Soit t un tableau de nombre entiers distincts et trié en ordre croissant. On désire résoudre le problème suivant :

Existe-t-il un indice i tel que $t[i] = i$?

Écrivez un algorithme qui vérifie si un tel index existe dans t. Votre algorithme doit s'exécuter dans le pire des cas en $\log n$ où n est la taille du tableau.

Exercice 3

Pour les équations de récurrences ci-bas, donnez une fonction f telle que $\tau \in \Theta(f)$

a) $\tau(n) = 3\tau(\frac{n}{5}) + \sqrt{n}$
b) $\tau(n) = \tau(n-8) + 2n^2$
c) $\tau(n) = 9\tau(\frac{n}{3}) + n^3$
d) $\tau(n) = \tau(\frac{n}{4}) + \log^4 n$
e) $\tau(n) = 2\tau(n-2) + 2^n$
f) $\tau(n) = 7\tau(\frac{n}{4}) + n \log n$

g) $\tau(n) = \tau(n-1) + 3^n$
h) $\tau(n) = 3\tau(\frac{n}{2}) + n^{\frac{5}{2}}$
i) $\tau(n) = 2\tau(\frac{n}{2}) + n^2$
j) $\tau(n) = 5\tau(\frac{n}{3}) + n\log^2 n$
k) $\tau(n) = 8\tau(\frac{n}{2}) + \sum_{i=0}^{n} 4^i$
l) $\tau(n) = 2\tau(\frac{n}{2}) + \sqrt{n}$
m) $\tau(n) = \tau(n-2) + n^2$
n) $\tau(n) = 8\tau(\frac{n}{4}) + n^3$
o) $\tau(n) = 9\tau(\frac{n}{3}) + n^2$
p) $\tau(n) = 5\tau(\frac{n}{3}) + n\log^2 n$
q) $\tau(n) = \tau(n-1) + \log n$

Exercise 4

Le but de cet exercice est de résoudre des équations de récurrence de la forme,

$$\tau(n) = \tau(n-\beta) + \tau(\beta) + n$$

où n est un nombre naturel positif.

a) Résolvez cette équation de manière exacte dans le cas où β est égal à 1. Vous pouvez supposer que $\tau(1)$ est une constante connue.
b) Résolvez cette équation de manière exacte dans le cas où β est égal à 2. Vous pouvez supposer que $\tau(2)$ et $\tau(1)$ sont des constantes connues.
c) Pouvez-vous «deviner» la complexité de la solution lorsque β est un nombre quelconque, mais fixe ?

Chapitre 4

Diviser pour régner

Ce paradigme algoritmique se base sur le principe suivant :

- Diviser le problème en sous-problème (Diviser)
- Résoudre ces sous-problèmes indépendamment (Régner)
- Combiner les résultats obtenus dans la résolution des sous-problèmes (Combiner)

Ce principe est très utile lorsque la division des sous-problèmes ne s'entrelacent pas.

La complexité asymptotique d'un algorithme utilisant ce principe se formule sous la forme d'une équation de récurrence aux divisions finies.

4.1 Série 4

Exercice 1

Considérez le problème suivant : un tableau de taille n contient tous les nombre entiers de 0 à n **excepté un nombre**. Les éléments du tableau sont triés de façon croissante. Le problème consiste à trouver l'entier qui n'est pas dans le tableau (un nombre entre 0 et n). Voici quelques exemples :

n	tableau	entier manquant
5	$[0, 1, 3, 4, 5]$	2
5	$[1, 2, 3, 4, 5]$	0
7	$[0, 1, 2, 3, 4, 5, 6]$	7
8	$[0, 1, 2, 3, 5, 6, 7, 8]$	4

Voici un algorithme qui permet de résoudre ce problème.

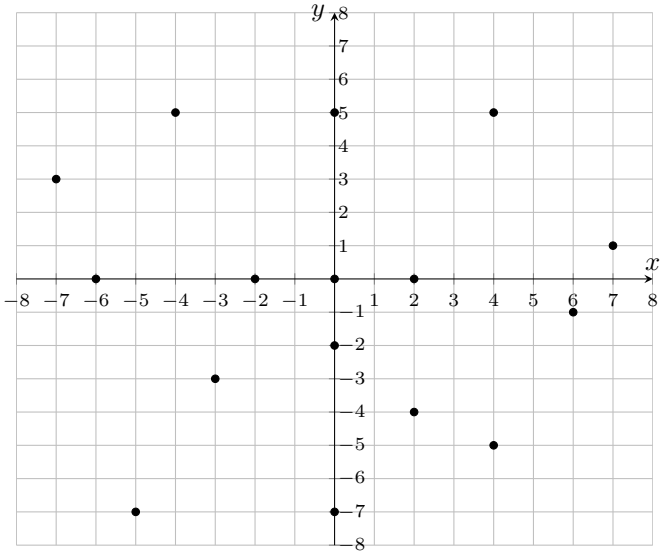
```
1  def entier_manquant(tab):
2      if not tab[0] == 0:
3          return 0
4      i = 1
5      reponse = 0
6      trouve = False
7      while i < len(tab) and not trouve:
8          if tab[i] = i:
9              i = i + 1
10         else
11             trouve = True
12     reponse = i
13     return i
```

a) Donnez le nombre d'exemplaires de problèmes possibles pour $n \geq 1$.
b) Existe-t'il une corrélation entre les indices du tableau et le nombre associé à la position i du tableau ?
c) Écrivez un algorithme utilisant le principe «diviser pour régner» et donnez la complexité temporelle de votre algorithme avec la notation Θ.
d) Si le tableau n'était pas trié, est-ce que l'algorithme écrit à la question précédente serait encore valide ?
e) Proposez un algorithme équivalent qui prend en compte la contrainte de la question précédente.

Exercice 2

Soit un tableau composé de points en 2D, on souhaiterait obtenir la plus petite distance entre deux points faisant partie du tableau. Pour cet exercise, vous avez accès à une fonction nommée **dist**(α, β) qui calcule la distance entre les points α et β en temps constant.

a) Considérez le plan suivant avec plusieurs points sur ce dernier.



Si la plus petite distance courante trouvée est celle entre les points $(-6, 0)$ et $(-7, 3)$, qui est d'environ $3,16$, si on utilise la distance euclidienne, quels seront les points où ils seraient plus probable de trouver une plus petite distance ?

Recommendentation : il peut être utile de dessiner entre chaque point une région pour lequel il y aurait une possibilité de trouver une plus petite distance que celle courante.

b) Donnez le nombre maximal de points qui pourront être potentiellement être la plus petite distance trouvé à date à partir d'un certain point.

Indice : Pour un point quelconque, dessinez la région auquel des points pourrait s'y retoruver et divisez cet espace en plus petit sous-espace en se basant sur une distance δ, la plus distance trouvé à date.

c) En supposant que le tableau soit trié en fonction de la coordonée x des points et de la coordonnée y en cas d'égalité, écrivez un algorithme utilisant le principe «diviser pour régner», qui prend en entrée un tableau de points avec les spécificités cités précédemment, et renvoit la plus petite distance entre deux points.

d) Si le tableau n'est pas trié, est-ce que l'algorithme écrit à la question précédente fonctionnerait ? Si elle ne fonctionne pas, donnez une modification pour que l'algorithme puisse s'exécuter correctement ainsi que sa

complexité temporelle.

Exercice 3

Nous voulons résoudre le problème qui consiste à déterminer s'il existe une valeur x telle que

$$|\{i|T[i] = x\}| \geq \frac{n}{2}$$

où T est un tableau composé d'un type de données nommé *Objet*.

a) Supposons qu'on a disposition des fonctions permettant de comparer ce type de donnée, décrivez un algorithme dans $\mathcal{O}(n \log n)$ qui résout ce problème si on considère *Objet* comme étant l'Entier (donc *Tab* est un tableau d'entiers).

b) Considérons le cas général où *Objet* est un type de données quelconque et supposons que seule l'opérateur booléen de l'égalité (noté $=$) est définit pour le type *Objet*, c'est-à-dire, l'opérateur $=$ permet donc de déterminer si deux valeurs de type *Objet* sont égales ou non (vous n'avez donc pas le droit aux opérations $<, >, \leq, \geq$). Appliquez le principe «diviser pour régner» pour décrire un algorithme dans $\mathcal{O}(n \log n)$ qui résout le problème pour le type *Objet*.

Exercice 4

Considérez l'algorithme suivant représentant une variante de l'algortithme FFT

```
1  def fft(tab):
2      if len(tab) == 1:
3          return tab
4      moitie = len(tab) // 2
5      w_n = exponentiation((-2. * pie) / (len(tab)) # ω_n^(-2iπ/n)
6      w = complex(1.0, 0.0)
7      pair = [complex(0.0, 0.0)] * milieu
8      impair = [complex(0.0, 0.0)] * milieu
9      fft(pair)
10     fft(impair)
11     k = 0
12     resultat = [complex(0.0, 0.0)] * len(tab)
13     while k < milieu:
14         resultat[k] = pair[k] + w_n * impair[k]
15         resustat[k + milieu] = pair[k] - w_n * impair[k]
16         w = w * w_n
17         i = i + 1
18     return resultat
```

L'algorithme travaille avec des nombres complexes qui sont formalisés de la manière suivante : un nombre complexe z est composé de deux composants représentable de la manière suivante

$$z = a + bi \tag{4.1}$$

La constante i est le nombre imaginaire et a la propriété que $i^2 = -1$. On note aussi que $z \in \mathbb{C}$.

La fonction EXPONENTIATION calcule l'exponentiation d'un nombre imaginaire en utilisant la formule d'Euler :

$$e^{ix} = \cos x + i \sin x \tag{4.2}$$

Cet algorithme a de belles propriétés et est beaucoup appliqué notamment dans le domaine du traitement du signal, du traitement d'image ou bien de l'algèbre pour ne citer que certains.

a) Formalisez l'équation de récurrence qui a la forme suivante :

$$\tau(n) = \alpha\tau(\frac{n}{\beta}) + f(n)$$

tout en donnant les constantes α et β tels que $a > 0, \beta > 1, \alpha, \beta \in \mathbb{N}$ ainsi que la définition de la fonction f tel que $f : \mathbb{N} \to \mathbb{N}$.

b) À l'aide du théorème général, donnez une fonction $g : \mathbb{N} \to \mathbb{R}^+$ telle que $\tau \in \Theta(g)$.

Exercice 5

Soit *tab*, un tableau composé d'entier distincs triés en ordre **décroissant** pour lequel on voudrait créer un arbre binaire de recherche à partir de ce dernier dont sa hauteur est minimale. Pour cet exercise, considérez que vous avez accès à la fonction ABR(n, T_1, T_2) qui crée un arbre dont la valeur de la racine est n, son enfant gauche est T_1 et son enfant droite est T_2.

a) Écrivez un algorithme permettant de résoudre le problème en utilisant le principe «diviser pour régner».
b) Donnez l'équation de récurrence de votre algorithme.
c) À l'aide du théorème général, donnez la complexité de votre algorithme.

4.2 Extra

Exercice 1

Considérez une matrice telle que chaque élément d'une ligne soit trié en ordre croissant et chaque élément d'une colonne est trié en ordre croissant.

Par exemple, une instance valide de matrice valide est la suivante :

$$M = \begin{bmatrix} 1 & 4 & 7 & 8 \\ 4 & 5 & 9 & 10 \\ 8 & 10 & 14 & 15 \end{bmatrix}$$

Écrivez un algorithme permettant de vérifier si un élément x est présent dans la matrice M en utilisant le principe «diviser pour régner».

Exercice 2

Une matrice booléenne M est une matrice composée seulement de 1 ou de 0.

Le produit matriciel entre deux matrices booléennes, dont l'opérateur est noté $\odot$, est une matrice dont le résultat de chaque élément entre le produit des matrices A et B est le suivant :

$$c_{i,j} = \bigvee_{k=1}^{m} a_{i,k} \wedge b_{k,j}$$

où $c_{i,j}$ est un élément dans la matrice résultante à la coordonée (i, j), $a_{i,k}$ l'élément à la coordonnée (i, k) de la matrice A, $b_{k,i}$ l'élément à la coordonnée (k, j) de la matrice B et m la taille d'une ligne de la matrice A ou la taille d'une colonne dans la matrice B.

a) Donnez le produit matriciel $C = A \odot B$, si les matrices A et B sont les suivantes :

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad B = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

b) En se basant sur la définition mathématique, donnez la complexité temporelle d'un algorithme se basant seulement sur cette définition.

c) Proposez un algorithme permettant de faire le produit matriciel de deux matrices booléennes en utilisant le principe «diviser pour régner».

Chapitre 5

Programmation dynamique

La programation dynamique est une technique utilisée pour les algortihmes d'optimisations. Si les sous-problèmes se chevauchent, la programmation dynamique devient un bon choix pour résoudre un problème.

Plutôt que de partir du problème initial est de le résoudre en divisant le problème en sous-problème, la programmation dynamque part d'un cas de base et adapte le problème lors de la remontée vers la solution finale.

L'approche utilisé par ce paradigme algorithmique est dite *bottom-up* au lieu des approches comme l'approche «diviser pour régner» qui utilise une approche *top-down*.

5.1 Série 5

Exercice 1

Trouvez la plus longue sous-suite commune entre les mots **algorithmie** et **isomorphisme**.

Exercice 2

Trouvez la distance de Levenshtein et un alignement pour les mots **scinder** et **cisailler**. Assumez que le coût pour l'insertion, la suppression et le changement de caractère est de 1.

Exercice 3

Trouvez la distance de Levenshtein pour les séquences **CGATAGAT** et **GTCCAG** si le coût des différentes opérations sont les suivantes :

— Insertion : 3
— Suppression : 2
— Changement de caractère :

	A	G	T	C
A	0	2	1	3
G	2	0	2	1
T	1	2	0	2
C	3	1	2	0

Exercice 4

Un voyageur souhaitant se rendre à une ville v_n à partir de la ville v_1. Pour cela, il a décidé de prendre sa voiture, qui utilise de l'essence, pour se rendre à la ville v_n . Sachant que la distance pour se rendre à une ville v_p à partie d'une ville v_w n'est pas nécessairement équivalente à la consommation d'essence pour se rendre à la ville v_p et ,bien évidemment, il ne veut pas retourner à une ville précédente. Il aimerait savoir quel serait l'itinéraire le plus court qui limiterait sa consommation d'essence. Les matrices de coûts D et les matrices de consommation d'essence C sont représentées ainsi :

$$D, C = \begin{bmatrix} 0 & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ \infty & 0 & a_{2,3} & \cdots & a_{2,n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \infty & \infty & \infty & \cdots & a_{m,n} \end{bmatrix}$$

a) Donnez le schéma dynamique de ce problème.

Indice : $\text{argmin}_{i=m}^{n} a_i$ cherche le minimum entre les éléments a_m à a_n.

b) On vous basant sur le schéma dynamique trouvé à la question précédente, écrivez un algorithme utilisant la **programmation dynamique** qui prend au moins en entrée les éléments suivants :

1. La matrice des distances entre les villes D.
2. La matrice de consommation d'essence C

Exercice 5

Soit le problème qui cherche le plus grand carré de 1 dans une matrice carrée A dont les coefficients valent 0 ou 1.

Par exemple, dans la matrice suivante :

$$\begin{Bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{Bmatrix}$$

l'algorithme devrait renvoyer 3.

Donnez un algorithme de programmation dynamique qui prends en entrée une matrice A respectant les contraintes du problème et renvoie la largeur maximum k d'un carré de 1 dans A.

5.2 Extra

Exercice 1

Trouvez la plus longue sous-suite commune entre les mots **constitutionnellement** et **constipation**.

Exercice 2

Trouvez la plus longue sous-suite commune entre les mots **touchable** et **disponible**.

Exercice 3

Trouvez la distance de Levenshtein et un alignement pour les mots **titubiler** et **tisane**.

Exercice 4

Trouvez la distance de Levenshtein et un alignement pour les mots **actuellement** et **couramment**.

Exercice 5

Les nombres de Catalan sont décrivables ainsi :

$$C_n = \begin{cases} 1 & \text{si } n = 0 \\ \sum_{i=1}^{n} C_{i-1} C_{n-i} & \text{sinon} \end{cases} \tag{5.1}$$

Écrivez un algorithme de programmation dynamique permettant de calculer le énième nombre de Catalan.

Exercice 6

Considérez le problème, suivant pour lequel dans un batiment de n étages avec en possession k oeufs, on souhaiterait connaître quel est l'étage maximal de ce batiment auquel un oeuf ne se brisera pas à l'impact. Cet étage se nommera l'étage critique.

Les règles concernant les oeufs sont les suivantes :

- Si l'oeuf se casse, alors tout autre lancer d'oeuf à une étage supérieur à cet étage brisera aussi les oeufs restants.
- Si l'oeuf ne casse pas, alors tout étage inférieur à celui lancer ne briseront pas d'oeuf. L'oeuf lancé peut être réutilisé dans ce cas-ci.

Pour les trois premières questions suivantes, on assumera que $k = 2$.

a) Si on suppose que le nombre d'étages est de 36, donnez l'étage critique.
b) Donnez le schéma de programmation dynamique.
c) Écrivez un algorithme de programmation dynamique qui se base sur le schéma dynamique donné à la question précédente.
d) Si on supposez qu'on a disposition k oeufs à disposition, donnez une généralisation de cette formule.

Chapitre 6

Algorithmes gloutons

Les critères d'un algorithme glouton sont les suivantes :

- L'algorithme est «myope», soit il ne considère que le meilleur résultat localement.
- L'algorithme pioche dans un ensemble de candidats possiblement meilleur et si celui-ci est pire que la meilleure solution courante, alors il ne sera plus considérer pour tout le long de l'algorithme.

Un algorithme glouton n'est pas nécessairement optimal, mais peut être proche de la/des solution(s) optimale(s).

6.1 Série 6

Exercise 1

Soit le tableau suivant contient des lettres avec leurs fréquences suivantes :

Lettre	a	b	c	d	e	f	g	h	i
Fréquences	26	24	13	12	7	6	5	4	3

a) Construisez un code le longueur minimale fixe pour ces lettres et donnez sa longueur.

b) Construisez le code de Huffman pour ces lettres et donnez sa longueur.

Exercise 2

Dans quel ordre il faut effectuer les tâches pour minimiser la somme des pénalités des tâches en retard ? On suppose que les tâches ont une durée de 1 et que les échéances et les pénalités sont données dans le tableau suivant :

Tâche	1	2	3	4	5	6	7	8
Échéance	3	2	2	3	4	5	1	7
Pénalité	95	85	55	60	75	45	40	30

Exercise 3

Nous considérons une variante du problème de la petite monnaie. Il s'agit de déterminer s'il existe des entiers naturels x_i tels que

$$\sum_{i=1}^{n} p_i x_i = P$$

où les p_i, les x_i et P ont la même signification que dans les diapositives du cours. Nous supposons que les p_i sont en ordre croissant. Notez que si p_1 n'est pas égal à 1, l'équation ci-dessous peut ne pas avoir de solution. En d'autres termes, il peut s'avérer impossible de rendre la monnaie, même si on dispose de quantités illimitées de chaque type de pièce. Pour rendre la monnaie, on applique l'algorithme glouton qui consiste à choisir d'abord les pièces de plus grande valeur. S'il n'est pas possible de rendre la monnaie qui reste après avoir considéré $p_n, p_{n-1}, \cdots, p_2, p_1$ l'algorithme glouton conclut qu'on ne peut pas rendre la monnaie.

a) L'algorithme glouton donne-t-il la bonne réponse si $n = 3, p_1 = 2, p_2 = 5, p_3 = 10$ et $P = 6$?

b) L'algorithme glouton donne-t-il la bonne réponse si $n = 3, p_1 = 2, p_2 = 6, p_3 = 8$ et $P = 12$?

c) Supposons maintenant que, comme dans la «vrai vie», le nombre de pièces de chaque pièce soit limité. Écrivez l'algorithme qui prend en compte ces contraintes. Assumez que l'algorithme prends en argument un couple telles (p_i, b_i) où p_i représente la valeur de la pièce p_i et b_i la quantité de pièce p_i de disponible.

Exercise 4

Soit le problème du palindrome suivant. Étant donné une liste de caractères en miniscules L, quel est le plus long palindrome qui peut être formé par les caractères de la liste L ?

Par exemple, si $L =$ **['a', 'b', 'c', 'c', 'c', 'c', 'd', 'd']** alors un des plus longs palindromes que l'on peut construire est *dccacd*, dont la longueur est de 7.

Écrivez un algorithme glouton qui prend en paramètre une liste de caractères L et qui retourne un entier représentant la taille du plus long palindrome qu'on peut construire avec les caractères de la liste L.

6.2 Extra

Exercise 1

Soit le tableau suivant contient des lettres avec leurs fréquences suivants :

Lettre	a	b	c	d	e	f	g	h
Fréquences	12	16	13	12	18	17	15	15

a) Construisez un code le longueur minimale fixe pour ces lettres et donnez sa longueur.

b) Construisez le code de Huffman pour ces lettres et donnez sa longueur.

Exercise 2

Soit le tableau suivant contient des lettres avec leurs fréquences suivants :

Lettre	a	b	c	d	e	f	g	h	i
Fréquences	20	8	9	10	12	11	14	9	7

a) Construisez un code le longueur minimale fixe pour ces lettres et donnez sa longueur.

b) Construisez le code de Huffman pour ces lettres et donnez sa longueur.

Exercise 3

Dans quel ordre il faut effectuer les tâches pour minimiser la somme des pénalités des tâches en retard ? On suppose que les tâches ont une durée de 1 et que les échéances et les pénalités sont données dans le tableau suivant :

Tâche	1	2	3	4	5	6	7	8
Échéance	3	2	2	3	4	5	1	7
Pénalité	55	45	65	80	85	95	60	70

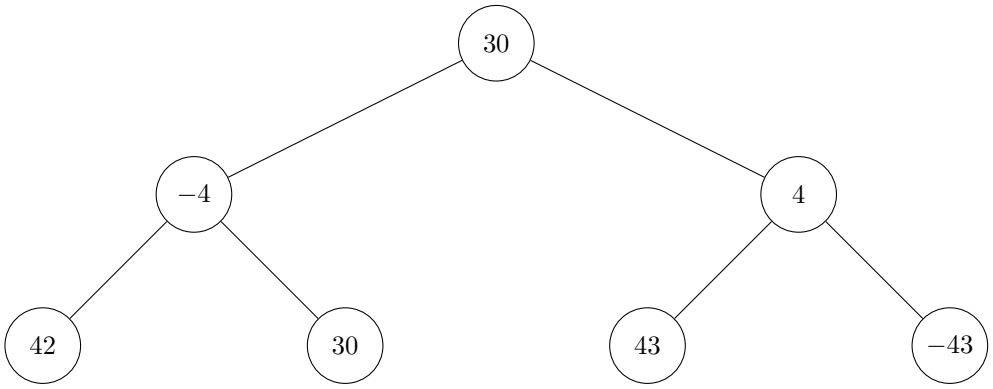
Exercise 4

Dans quel ordre il faut effectuer les tâches pour minimiser la somme des pénalités des tâches en retard ? On suppose que les tâches ont une durée de 1 et que les échéances et les pénalités sont données dans le tableau suivant :

Tâche	1	2	3	4	5	6	7	8
Échéance	3	2	2	3	4	5	1	7
Pénalité	55	45	65	80	85	95	60	70

Exercice 5

Considérez le problème suivant auquel à partir d'un arbre binaire complet pour lequel chaque noeud possède une entier $n \in \mathbb{Z}$. On souhaiterait, trouvez le chemin partant de la racine jusqu'à une feuille telle que la sommation soit maximalle. Par exemple, pour l'arbre suivant :



La chemin donnant la sommation maximale est 77. En effet, le chemin à prendre serait en partant de la racine $30 + 4 + 43 = 77$, soit le chemin droite suivi de la gauche.

Pour les besoins de l'algorithme, vous avez accès aux fonctions suivantes s'exécutant en temps constant :

- OBTENIR_GAUCHE(N) : renvoie l'enfant de gauche du noeud N. La fonction renvoie *None* si le N est une feuille.
- OBTENIR_DROITE(N) : renvoie l'enfant de droite du noeud N. La fonction renvoie *None* si le N est une feuille.
- OBTENIR_CONTENUE(N) : donne le contenue du noeud N.

a) Écrivez un algorithme glouton permettant de résoudre le problème.
b) Si votre algorithme est optimal, prouvez le. Dans le cas contraitre, donnez un cas où l'algorithme ne donne pas la solution optimale.

Chapitre 7

Algorithmes sur les graphes

Rappel : un graphe noté G est un ensemble composé de deux ensembles :

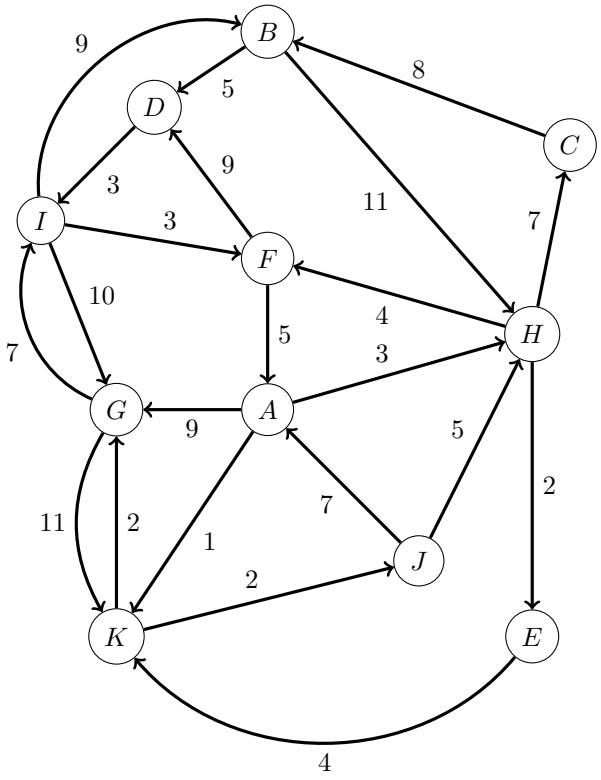
- S : l'ensemble de sommets que le graphe G contient.
- A : l'ensemble des arrêts que le graphe G contient. Un arrêt relie deux points entre eux qui peuvent être le même sommet.

Dû qu'un graphe dépend de deux ensembles de deux ordre gradeurs différentes, alors à moins qu'un algorithme ne dépend qu'un des deux ensembles, alors la complexité algorithmique utilisant les graphes dépend de $|S|$ et de $|A|$.

7.1 Série 7

Exercice 1

Considérez le graphe suivant :



a) Effectuez une recherche en profondeur si le sommet de départ est A
b) Effectuez une recherche en largeur si le sommet de départ est A
c) Effectuez l'algorithme de Prim si le sommet de départ est A
d) Effectuez l'algorithme de Kruskal

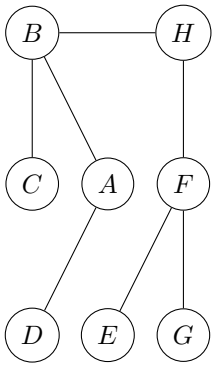
Exercice 2

Soit un arbre, noté T, est aussi un graphe ayant les propriétés suivantes :

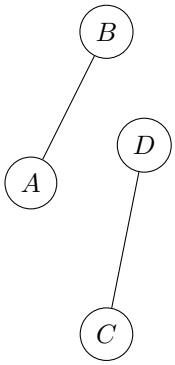
1. $|A| = |S| - 1$
2. Le graphe est acyclique, c'est à dire qu'il est dépourvu de cycle

a) Dites si c'est les graphes suivants sont des arbres :

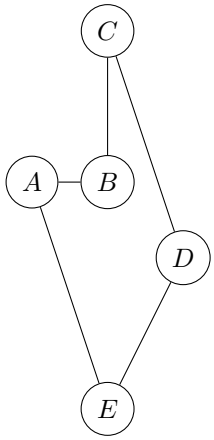
Graphe 1



Graphe 2



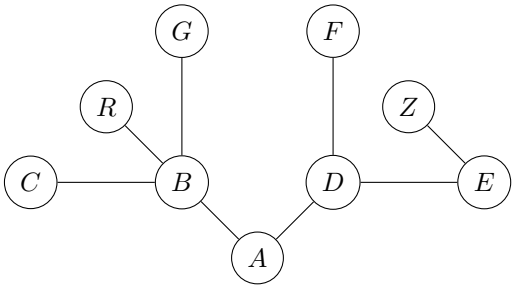
Graphe 3



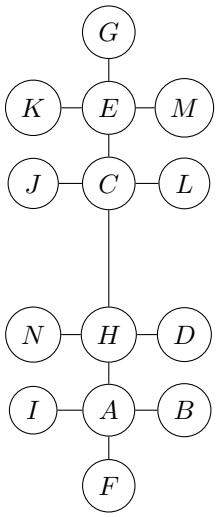
b) Un moyen de déterminer si un graphe est un arbre et de vérifier s'il existe

un ou deux centres. Pour ce faire, une approche possible est d'«enlever» les sommets qui sont de dégré 1 et réduire le nombre de degré des autres sommets dont leur degré est supérieur à 1 et refaire ce même processus jusqu'à qu'il ne reste plus de sommets de degré 1. Un arbre aura toujours au moins un centre et au plus 2 centres. En considérant les graphes suivants, déterminez son/ses centre(s).

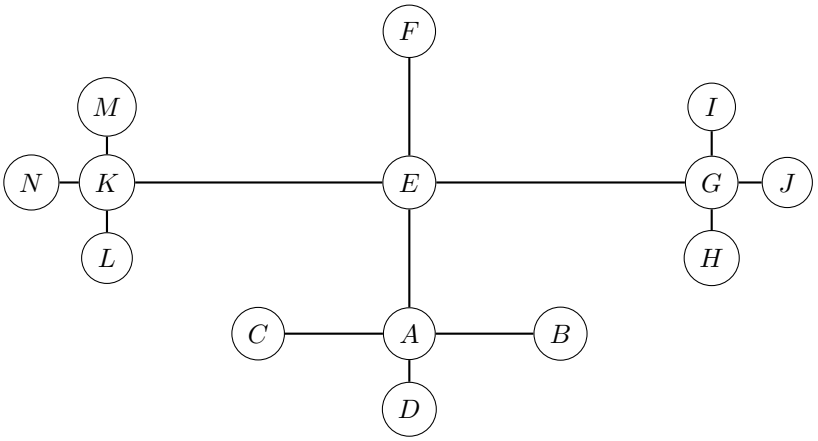
Graphe 1



Graphe 2



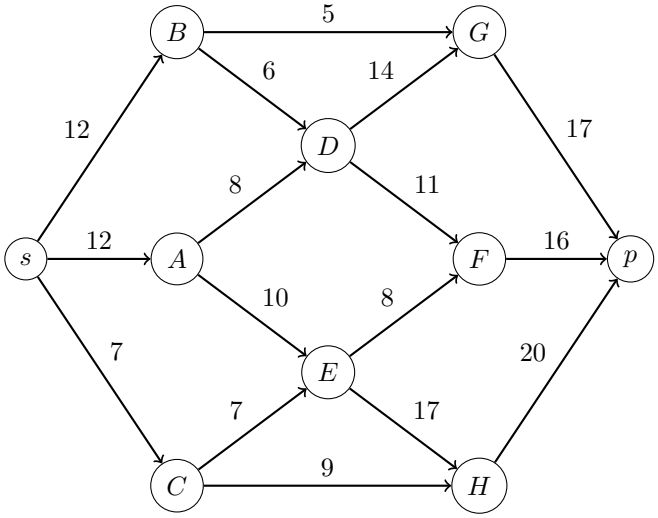
Graphe 3



c) Écrivez un algorithme qui prend un graphe G et vérifie si ce graphe est un arbre.

Exercice 3

Considérez le graphe suivant :

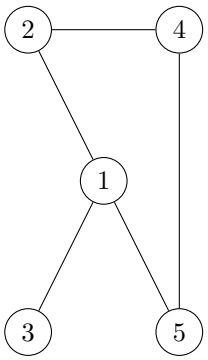


a) Trouvez le flot maximal en utilisant la variante **Edmond-Karp**.
b) Trouvez le flot maximal en utilisant **une recherche en pronfondeur**.

7.2 Extra

Exercice 1

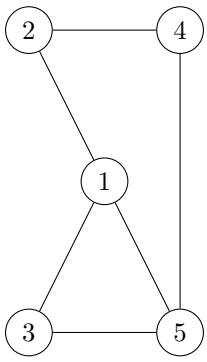
Une articulation est un sommet pour lequel si on enlève ce sommet d'un graphe non orienté, alors il apparation de plusieurs sous-graphes non connectés entre eux. En d'autres termes, le graphe sera déconnecté. Par exemple, le graphe suivant



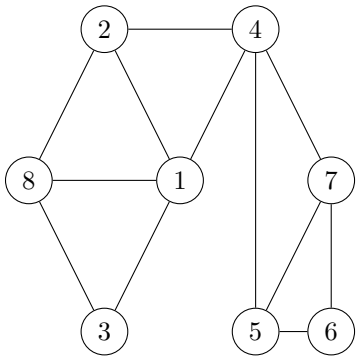
possède un point d'articulation qui est le sommet 1.

a) Pour les graphes suivants, déterminez si les graphes suivants possèdent une articulation.

1.



2.



b) Proposez un algorithme qui étant donnée un graphe non orienté, vérifie si ce dernier possède un point d'articulation.

Indice : une approche possible est de construire dynamiquement un graphe en forme d'arbre et de vérifier s'il existe des arrêts qui sont des *back-edges*.

Chapitre 8

Analyse amortie

- Analyse classique : la complexité asymptotique est déterminé par l'instruction la plus coûteuse sur une opération quelconque.
- Analyse amortie : la complexité amortie est déterminé par la moyenne des coûts sur un ensemble de n séquences.

Les méthodes d'analyse possibles lors d'une analyse amortie :

- Méthode de l'agrégat : déterminer le coût moyen d'une séquence de n opérations.
- Méthode du comptable : supposer que le coût amortie est équivalent à un coût y et vérifier si la différence entre le coût amortie cumulé et le coût réel est ≥ 0.
- Méthode du potentiel : soit Φ_d, l'état de la structure de donné après d opérations, le coût amortie est donnée par

$$\sum_{i=1}^{n} ca(i) = \sum_{i=1}^{n} cr(i) + \Phi_i - \Phi_{i-1}$$

.

8.1 Série 8

Exercise 1

Considérez une structure de données dont le coût des opérations sont les suivantes :

$$cr(i) = \begin{cases} i+1 & \text{si i est une puissance de 4} \\ 1 & \text{sinon} \end{cases}$$

a) Effectuez l'analyse amortie de cette structure de données en utilisant la méthode de l'agrégat
b) Effectuez l'analyse amortie de cette structure de données en utilisant la méthode du comptable
c) Effectuez l'analyse amortie de cette structure de données en utilisant la méthode du potentiel

Exercise 2

Une manière de représenter un «dictionnaire» est la suivante : on a un ensemble de tableaux qui ont une taille équivalente à une puissance de 2. Ces derniers sont soient vide ou pleine. Dans le cas où elles sont pleines, elles sont triées. En revanche, malgré qu'il y a plusieurs tableaux triés, ces derniers ont aucune relation entre eux.

Voici un exemple avec d'un «dictionnaire» à 11 éléments :

R_1 [6]

R_2 $[2, 10]$

R_3 Vide

R_4 $[-1, 2, 5, 7, 8, 9, 10, 25]$

Pour savoir quels sont les tableaux qui sont pleins, il faut regarder la représentation binaire du nombre d'élément présents. Ici,

$$(11)_{10} = (1011)_2$$

les puissances de 2 qui sont marqués à 1 dans la rerprésentation binaire du nombre d'éléments. Ici, les puissances 0, 1 , 3 signifient qu'il y a un tableau plein des tailles suivantes : 1, 2, 8. Lors de l'insertion d'un nouvel élément, les deux situations suivantes peuvent se produisent :

1. S'il n'y a pas de tableau de taille 1 qui est déjà présente, alors on crée seulement un tableau de taille 1.

2. Dans le cas où un tableau de taille 1 est déjà présent, alors on fusionne ces deux tableaux en les mettant les éléments dans l'ordre croissant. Dans le cas de la fusion de deux tableaux triés (de même taille dans ce cas-ci), il est possible de les fusionner en $\Theta(p)$ où p est le nombre d'éléments des deux tableaux. Ensuite, dû que le nouveau tableau a doublé en taille, il faut vérifier s'il existe un tableau de même taille que le nouveau tableau. Si c'est le cas, on répéte le même processus jusqu'à que le tableau crée d'une taille 2^c, où c est une constante, soit le seul tableau de cette taille.

Par exemple, si on ajoute l'élément 3 dans la structure de tout à l'heure, on obtient le résultat suivant :

R_1	Vide
R_2	Vide
R_3	$[2, 3, 6, 10]$
R_4	$[-1, 2, 5, 7, 8, 9, 10, 25]$

Or, $(12)_{10} = (1100)_2$ ce qui signifie qu'il y a seulement un tableau pour les tailles suivantes : 4 et 8. Pour rechercher un élément, il est possible d'effectuer une fouille dichotomique sur chaque tableau jusqu'à un des tableaux contient l'élément recherché ou qu'aucune ne la continne.

a) Donnez la complexité dans le pire des cas pour l'insertion d'un nouvel élément et sa complexité en amortie pour insérer un nouvel élément.
b) Donnez la complexité de la recherche d'un élément dans cette structure de donnée.

8.2 Extra

Exercice 1

Supposons que les coûts d'une opération sur une certaine structure de donnée soit la suivante :

$$cr(i) = \begin{cases} \frac{i}{8} + 1 & \text{si } i \mod 16 = 0 \\ 1 & \text{sinon} \end{cases} \tag{8.1}$$

Donnez la complexité amortie de cette structure de donnée.

Exercice 2

En classe, vous avez certainement vu le monceau binomial qui a ces complexités en amortie suivantes selon n, le nombre d'élément dans le tas, :

Opération	Complexité amortie
Insertion d'un élément	$\Theta(1)$
Extraction du minimum/maximum	$\Theta(\log n)$
Fusion de deux files binomiaux	$\Theta(\log n)$

Or, il est possible d'améliorer l'insertion d'un élément et la fusion de deux files binomiaux en garantissant qu'il s'exécute en temps constant tout en gardant la complexité amortie de l'extraction du minimum et du maximum que le monceau binomial offre. Pour cela, les opérations se procédent ainsi

- L'insertion d'un élément : insérer l'élément dans la file sans effectuer aucune fusion et mettre à jour le minimum ou le maximum si nécessaire.
- Fusion de deux files binomiaux : mettre tous les arbres d'une file dans l'autre file et mettre à jour le mininum ou le maximum si nécessaire sans fusionner les arbres entre eux. Note : avec des listes chainées, il est possible de fusionner deux listes chainées en temps constant.
- Extraction du minimum/maximum : retirer le minimum et le maximum de la file et ensuite fusionner tous les arbres de même degré et refaire le même processus pour les arbres nouvellement formés jusqu'à qu'aucune fusion ne soit possible.

On nommera cette variante du monceau binomial comme étant un monceau binomial paresseux. Pour les exercices suivants, on va assumer que le monceau est un monceau min.

a) Afin de se familiariser avec cette variante, effectuez les opérations suivantes (dans l'ordre) et donnez à chaque étape l'état de la file :

1. Insertion de 7
2. Insertion de 3
3. Insertion de 4
4. Insertion de 10
5. Insertion de 5
6. Suppression du minimum

7. Insertion de 40
8. Insertion de 57
9. Suppression du minimum
10. Insertion de 23
11. Suppression du minimum

b) Effectuez une analyse amortie de cette structure de donnée.

Exercice 3

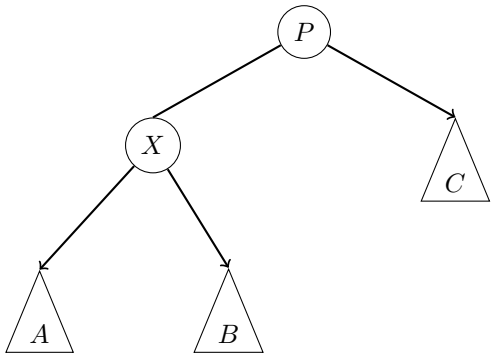
Dans le cours INF3105, vous avez certainement vus les arbres de recherches qui s'équilibrent eux-mêmes (arbres AVL, rouge-noir et arbre-B). Ces arbres garantissent une recherche, une insertion et une suppression en temps logarithmique. Cependant, ces arbres souffrent d'un défaut, il est possible que les éléments les plus recherchés se trouvent en bas de l'arbre se qui peut devenir coûteux. Une structure de donnée quiu permet de mettre les éléments les plus recherchés de l'arbre le plus proche de la racine est un arbre évasé (splay tree).

Un arbre évasé à la propriété d'utiliser un type d'opération spéciale nommée *évasement*. Ce type d'opération peut être une des trois des sous-opérations suivantes :

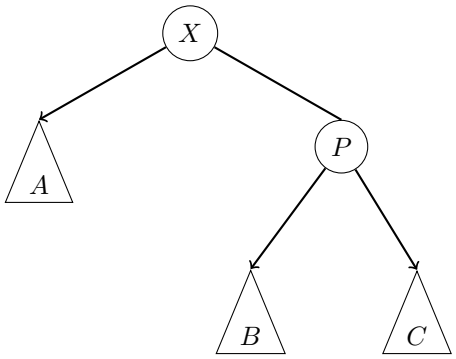
- *Zig*
- *Zig-zag*
- *Zig-zig*

Important : il est possible de découler les opérations dans le cas où on fait ces opérations dans l'autre sens.

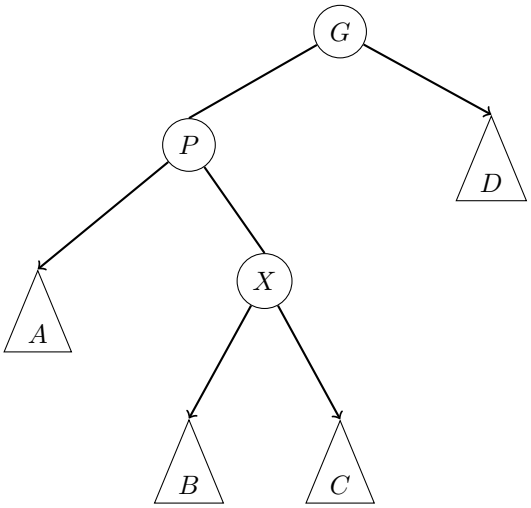
L'opération *zig* consiste à faire une rotation simple pour les noeuds X et P si P est le parent de X et que P est la racine de l'arbre. Par exemple, l'arbre suivant



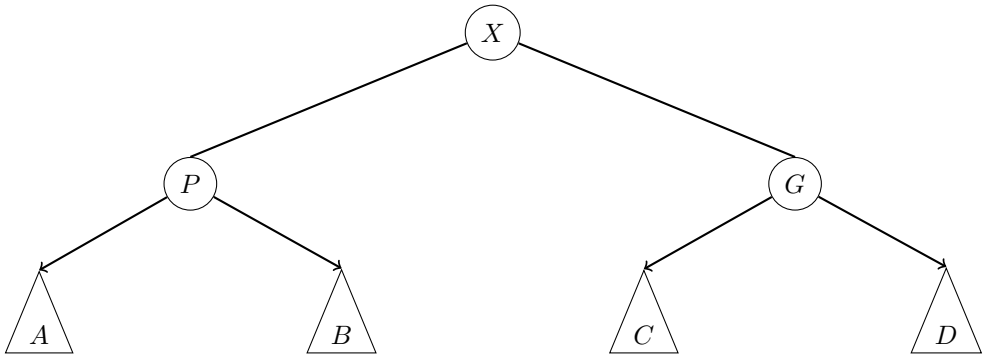
devient après une opération *zig* l'arbre suivant



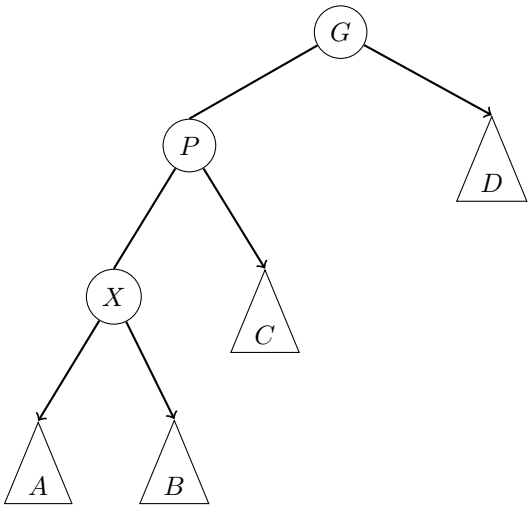
L'opération *zig-zag* consiste à faire une double rotation (droite-gauche ou gauche-droite) comme s'il s'agissait d'un arbre AVL.



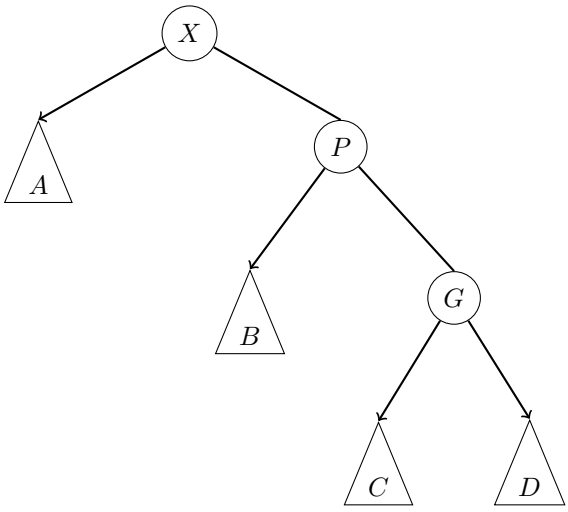
devient l'arbre suivant après cette opération



L'opération *zig-zig* consiste à faire une double rotation du style droite-droite ou gauche-gauche.



devient l'arbre suivant après l'opération suivant :



Les opérations de bases se font de la sorte :

- Insertion d'un nouvel élément : il faut insérer l'élément comme si c'était un arbre de recherche binaire classique et ensuite effectué les opérations précisé ci-haut jusqu'à que le nouvel élément soit la racine de l'arbre.
- Suppression d'un élément : on doit effectuer une suite d'opérations d'évasion jusqu'à l'élément à supprimer soit la racine de l'arbre et c'est dans ce cas-ci qu'on supprime l'élément. Ensuite, il faut évaser soit le plus grand élément qui est inférieur à l'élément supprimer ou bien évaser le plus petit élément qui est supérieur à l'élément à supprimer.
- Chercher un élément dans l'arbre : la procédure reste la même, excepté que si l'élément recherché existe, alors il faut évaser l'élément trouvé.

Dans le cas où l'élément n'existe, alors il faut évaser le dernier élément qui fût comparé.

Pour les exercices suivants, on va considérer la première possibilité dans le cas de la suppression d'un élément.

a) Afin de se familiariser avec les opérations de cette structure de données, effectuez les opérations suivantes (en ordre) et donnez la forme de l'arbre :

1. Insertion de 10
2. Insertion de 40
3. Insertion de 30
4. Insertion de -4
5. Recherche de 30
6. Recherche de -10
7. Insertion de 50
8. Insertion de -32
9. Suppression de 10
10. Suppression de 30

b) Effectuez l'analyse amortie des opérations d'évasion.

c) En vous sur l'analyse amortie des opérations d'évasion, donnez la complexité amortie des opérations de la recherche d'un élément dans l'arbre, de l'insertion d'un nouvel élément et la suppression d'un élément.

Indice :

- Une idée de fonction de potentiel serait être capable de représenter l'état de cette structure de donnée de sorte que la fonction de potentielle donne une plus grande valeur lorsque un élément est proche de la racine et très faible dans le cas où il est très éloigné de l'arbre.
- Utiliser un logarithme pour représenter cette fonction de potentiel est très utile.

Chapitre 9

NP complétude

Problème N : Problème auquel il existe un algorithme en temps polynomial pour le résoudre.

Problème NP : Problème pour lequel il facile de vérifier si une solution x résout le problème.

Un des 7 problèmes du millénaire : $N \stackrel{?}{=} NP$.

NPC : problèmes les plus durs dans NP.

9.1 Série 9

Exercice 1

Soit l'intance SAT suivante :

$$\phi = ((\neg x_1 \wedge x_2) \rightarrow (x_2 \vee (x_1 \rightarrow x_5))) \rightarrow (x_4 \vee (\neg x_3 \wedge x_5))$$

a) Réduisez cette instance SAT en une instance 3-FNC-SAT
b) Avec l'instance 3-FNC-SAT trouvé à la question précédente, covertissez cette instance en une instance du problème *CLIQUE*
c) Trouvez au moins **deux** cliques qui satisfaient ce problème

Exercice 2

À partie des instances du problème 2-FNC-SAT, déterminez si les formules sont satisfaisables.

a) $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (x_4 \vee \neg x_3) \wedge (x_1 \vee \vee x_4) \wedge (x_5 \vee \neg x_4) \wedge (x_2 \vee \neg x_5)$
b) $(x_1 \vee x_2) \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (x_4 \vee \neg x_5) \wedge (x_5 \vee x_6) \wedge (\neg x_5 \vee x_1) \wedge (x_5 \vee \neg x_6)$

Exercice 3

Démontrez que la borne serré d'un algorithme de tri se basant sur la **comparaison** est $\Omega(n \log n)$.

9.2 Extra

Exercice 1

Soit l'intance SAT suivante :

$$\phi = ((\neg x_1 \wedge x_2) \rightarrow (x_2 \vee (x_1 \rightarrow x_5))) \wedge (x_4 \vee (\neg x_3 \wedge x_5))$$

Réduisez cette instance SAT en une instance 3-FNC-SAT.

Exercice 2

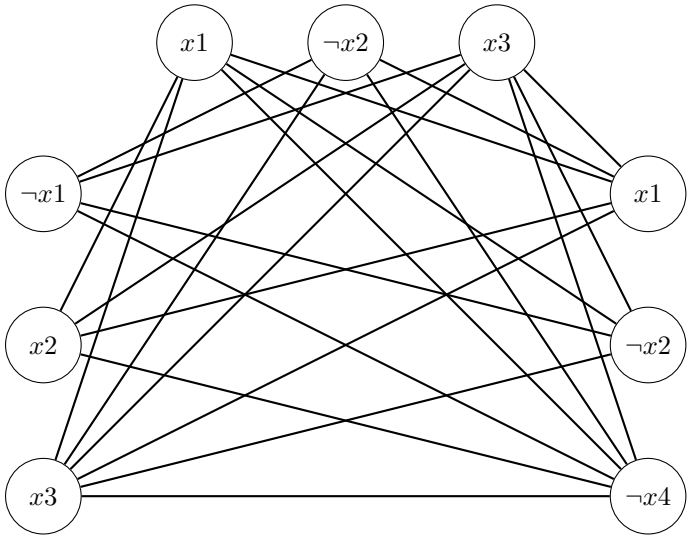
Soit l'intance SAT suivante :

$$\phi = ((\neg x_1 \wedge x_2) \rightarrow (x_2 \vee (x_1 \rightarrow x_5))) \rightarrow (x_4 \vee (\neg x_3 \wedge x_5))$$

Réduisez cette instance SAT en une instance 3-FNC-SAT.

Exercice 3

Trouvez une clique valide dans cet instance du problème clique.



Chapitre 10

Annexe

10.1 Composition

Cette partie comporte tout ce qui est rappel mathématique nécessaire pour pouvoir effectuer les exercices des sections précédentes. Il n'est pas obligatoire de faire ces exercices, mais il est **fortement** recommandé de les faires si ces rappels ne vous dient rien ou que les notions ne sont plus qu'un lointain souvernir.

Le début concerne les rappels mathématiques associés avec les exercices suivi de leur exercices.

10.2 Quelques propriétés logarithmiques

Soit $b \in \mathbb{R}^+ \backslash \{0, 1\}$ et $x > 0, y > 0$, alors voici une liste des propriétés sur les logarithmes :

- Valeur du logarithme de 1 :

$$\log_b(1) = 0$$

- Logarithme naturel :

$$ln(x) = \log_e(x)$$

- Logarithme avec même base et même coefficient :

$$\log_b(b^x) = x$$

- Produit dans un logarithme :

$$\log_b(xy) = \log_b(x) + \log_b(y)$$

- Quotient dans un logarithme :

$$\log_b(\frac{x}{y}) = \log_b(x) - \log_b(y)$$

- Puissance dans un logarithme :

$$\log_b(x^y) = y \log_b(x)$$

- Changement de base :

$$\log_b(x) = \frac{\log_k x}{\log_k b}$$

- Réduction des logs en puissance :

$$b^{\log_b(x)} = x$$

- Rotation des logs en puissance :

$$a^{\log_b(x)} = x^{\log_b(a)}$$

- Passage en forme de puisance :

$$\log_b x = y \rightarrow x = b^y$$

  Cette règle s'applique aussi dans l'autre sens.

10.3 Sommation

10.3.1 Propriétés des sommations

Soit $f : \mathbb{N} \rightarrow \mathbb{R}$

Sommation avec une constante

$$\sum_{i=s}^{n} = c \times f(i) = c \sum_{i=s}^{n} f(i)$$

Commutativité et associativité des sommations

Soit $g : \mathbb{N} \to \mathbb{R}$

$$\sum_{i=s}^{n} (f(i) \pm g(i)) = \sum_{i=s}^{n} f(i) \pm \sum_{i=s}^{n} g(i)$$

Fragmentation d'une sommation

Approche 1

$$\sum_{i=s}^{n} f(i) = \sum_{i=s}^{j} f(i) + \sum_{i=j+1}^{n} f(i)$$

Approche 2

$$\sum_{i=s}^{n} f(i) = \sum_{i=0}^{n} f(i) - \sum_{i=0}^{s-1} f(i)$$

10.3.2 Sommation particulière

Sommation d'une constante

Soit $n \in \mathbb{N}$ et $c \in \mathbb{R}$, alors

$$\sum_{i=1}^{n} c = nc$$

Somme arithmétique

Soit $n \in \mathbb{N}$, alors la somme des n premiers éléments est

$$\sum_{i=0}^{n} i = \frac{n(n+1)}{2}$$

Somme des nombres carrés

Soit $n \in \mathbb{N}$, alors

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

Théorème de Nicomachus

Soit $n \in \mathbb{N}$, alors

$$\sum_{i=0}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Somme géométrique

Soit $n \in \mathbb{N}$ et $a, r \in \mathbb{R}, r \neq 1$, alors

$$\sum_{i=0}^{n} ar^i = \frac{a(1-r^{n+1})}{1-r} = \frac{a(r^{n+1}-1)}{r-1}$$

Somme géométrique avec un multiplicateur variable

Soit $n \in \mathbb{N}$ et $r \in \mathbb{R}, r \neq 1$, alors

$$\sum_{i=0}^{n} ir^i = \frac{nr^{n+2} - (n+1)r^{n+1} + r}{(1-r)^2}$$

Somme télescopique

Soit $n \in \mathbb{N}$, alors

$$\sum_{i=0}^{n} a_{i+1} - a_i = a_{n+1} - a_0$$

10.3.3 Exercices

Exprimez les sommations suivantes sous la forme la plus simplifiée possible. Détaillez vos calculs.

Sommation 1

$$\sum_{i=k}^{m} \sum_{a=j}^{n} i \tag{10.1}$$

où k et j sont deux entiers tels que $k \leq m$ et $j \leq n$.

Sommation 2

$$\sum_{i=1}^{n} (2i-1) \tag{10.2}$$

Sommation 3

$$\sum_{j=0}^{n-1} j2^j \tag{10.3}$$

Sommation 4

$$\sum_{i=0}^{\beta}(i^2) - \sum_{j=0}^{\beta}(\beta - j) \tag{10.4}$$

Sommation 5

$$\sum_{d=n+1}^{n} d^4 \tag{10.5}$$

Sommation 6

$$\sum_{\alpha=1}^{k} \left(\frac{3}{2}\right)^{\alpha} \tag{10.6}$$

Sommation 7

$$\sum_{i=0}^{\log_3 n} \left(\frac{2}{3}\right)^{i} \tag{10.7}$$

Sommation 8

$$\sum_{i=0}^{\log_3 n} n\left(\frac{2}{3}\right)^{i} \tag{10.8}$$

Sommation 9

$$\sum_{k=2}^{p} k \tag{10.9}$$

Sommation 10

$$\sum_{p=0}^{n}(\sum_{p=0}^{n}5+\sum_{p=0}^{n}2) \tag{10.10}$$

Sommation 11

$$\sum_{\theta=0}^{n}(\theta-\sum_{\psi=0}^{m}2\psi) \tag{10.11}$$

Sommation 12

$$\sum_{i=1}^{\log n}\log(\frac{n}{2^i}) \tag{10.12}$$

Sommation 13

$$\sum_{s=0}^{n-1}s \tag{10.13}$$

Sommation 14

$$\sum_{k=2\sigma}^{m}k \tag{10.14}$$

où σ est un entier tel que $2\sigma \leq m$.

10.4 Démonstration

10.4.1 Démonstrations générale

Exercice 1

Démontrez que $\sqrt{2}$ est un nombre irrationnel.

Exercice 2

Démontrez que

$$\sum_{i=0}^{n}i=\frac{n(n+1)}{2} \tag{10.15}$$

Note : la démonstration doit être faite sans utiliser le principe d'induction.

Exercice 3

Soit $k, s \in \mathbb{N}$, $f : \mathbb{N} \to \mathbb{R}$ et $b > 0, b \neq 1, b \in \mathbb{R}$, démontrez que

$$\sum_{k=s}^{n} \log_b(f(k)) = \log_b(\prod_{k=s}^{n} f(k)) \tag{10.16}$$

Exercice 4

Démontrez que

$$4n^3 + n^2 \in \mathcal{O}(n^3) \tag{10.17}$$

Exercice 5

Démontrez que

$$\log n \notin \mathcal{O}(\log \log n) \tag{10.18}$$

10.4.2 Démonstration par induction

Exercice 1

Démontrez que

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \tag{10.19}$$

Exercice 2

Démontrez que

$$\sum_{i=0}^{n} ar^i = \frac{a(r^{n+1} - 1)}{r - 1} \tag{10.20}$$

Exercice 3

Les nombres de Lucas sont des nombres qui suit la récurrence suivante :

$$L_n = \begin{cases} 2 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ L_{n-1} + L_{n-2} & \text{sinon} \end{cases} \tag{10.21}$$

Démontrez que les nombres de Lucas respectent bel et bien cette récurrence.

Exercice 4

Soit la récurrence a_n qui est définie ainsi

$$a_n = \begin{cases} 1 & \text{si } n = 0 \\ 3 & \text{si } n = 1 \\ 2(a_{n-1}) + a_{n-2} & \end{cases} \tag{10.22}$$

Démontrez que cette récurrence renvoie tout le temps un nombre impair.

Exercice 5

Démontrez $(k+1)^3 - (k+1)$ est divisible par 3.

Corrigé

Série 1

Exercice 1

a) La première variante suppose que la plus petite puissance de 2 se trouve à la première case des tableaux A, B et C.

```
1   def additionner(A, B):
2       retenue = 0
3       C = [0] * (len(A) + 1)
4       i = 0
5       while i < len(A):
6           C[i] = (A[i] + B[i] + retenue) % 2
7           retenue = (A[i] + B[i] + retenue) // 2
8           i = i + 1
9       C[len(A)] = retenue
10      return C
```

La deuxième variante suppose que la plus petite puissance de 2 se trouve à la dernière position des tableaux A, B et C.

```
1   def additionner(A, B):
2       retenue = 0
3       C = [0] * (len(A) + 1)
4       i = len(A) - 1
5       while i >= 0:
6           C[i + 1] = (A[i] + B[i] + retenue) % 2
7           retenue = (A[i] + B[i] + retenue) // 2
8           i = i - 1
9       C[0] = retenue
10      return C
```

b) Dans les deux cas, une opération barométrique identifiable est la comparaison effectuée à la ligne 5, qui s'effectue $n + 1$ fois où n est la taille des tableaux A et B. Puisque cette ligne s'exécute en temps constant à chaque fois, la complexité temporelle de la fonction ADDITIONNER est $\mathcal{O}(n)$.

Exercice 2

a) Le tableau C doit maintenant avoir $2n$ éléments.

b) La première variante suppose que la plus petite puissance de 2 se trouve à la première case des tableaux A, B et C.

```
1   def multiplier(A,B):
2       C = [0] * (len(A))
```

```
3       decalage = 0
4       while decalage < len(A):
5           temp = [0] * (len(A) + decalage)
6           j = 0
7           while j < len(A):
8               temp[j + decalage] = A[j] * B[decalage]
9               j = j + 1
10          C = add_bin(temp, C)
11          decalage = decalage + 1
12      return C
```

La deuxième variante suppose que la plus petite puissance de 2 se trouve à la dernière case des tableaux A, B et C.

```
1   def multiplier(A,B):
2       C = [0] * (len(A))
3       decalage = 0
4       while decalage < len(A):
5           temp = [0] * (len(A) + decalage)
6           j = len(A) - 1
7           while j >= 0:
8               temp[len(temp) - decalage - (len(A) - j - 1) - 1] = A[j] * B[len(B) - decalage - 1]
9               j = j - 1
10          C = add_bin(temp, C)
11          decalage = decalage + 1
12      return C
```

c) Une opération barométrique possiblement identifiable pour évaluer la complexité de cet algorithme est l'instruction qui concerne l'addition de deux nombres binaires (ligne 10). Étant donné que la taille du tableau C doit avoir $2n$ éléments à la fin de l'algorithme et que ce dernier commence avec n éléments, le coût de cette instruction est de $\sum_{i=n}^{2n-1}(i+1)$ si on inclut le coût de l'assignation.

$$
\begin{aligned}
\tau(\text{MULTIPLIER}) &= \sum_{i=n}^{2n-1}(i+1) \\
&= \sum_{i=0}^{\psi} i + \sum_{i=0}^{\psi} 1 - (\sum_{i=0}^{n-1} i + \sum_{i=0}^{n-1} 1) && \text{posons } \psi = 2n-1 \\
&= \frac{\psi(\psi+1)}{2} + \psi + 1 - (\frac{n(n-1)}{2} + n) \\
&= \frac{4n^2-2n}{2} + 2n - 1 + 1 - \frac{n^2+n}{2} \\
&= \frac{4n^2-2n+4n-n^2-n}{2} \\
&= \frac{3}{2}n^2 + \frac{1}{2}n \in \mathcal{O}(n^2)
\end{aligned}
$$

Exercice 3

```
1   def contient_cycle(L):
2       lent = obtenir_tete(L)
3       rapide = obtenir_tete(L)
4       if rapide is None:
5           return False
6       cyclique = False
7       while not (rapide is None) and not (obtenir_suivant(rapide) is None) and not cyclique:
8           lent = obtenir_suivant(lent)
```

```
9           rapide = obtenir_suivant(rapide)
10          if rapide == lent:
11              cyclique = True
12      return cyclique
```

Exercice 4

a) En premier lieu, voici le nombre de fois où chaque opération est exécutée, ainsi que son coût :

Numéro de la ligne	Nombre de fois exécutée	Coût
1	1	$c_1 = k + 2$
2	1	$c_2 = n + 2$
3	1	c_3
4	$n + 1$	c_4
5	n	c_5
6	n	c_6
7	1	c_7
8	$k + 1$	c_8
9	k	c_9
10	k	c_{10}
11	1	c_{11}
12	$n + 1$	c_{12}
13	n	c_{13}
14	n	c_{14}
15	n	c_{15}
16	1	c_{16}

Le coût total de cette procédure est le suivant

$$\begin{aligned}
\tau(\text{TRICOMPTAGE}) &= c_1 + c_2 + c_3 + \sum_{i=0}^{n} c_4 + \sum_{i=0}^{n-1}(c_5 + c_6) + c_7 \\
&\quad + \sum_{i=0}^{k} c_8 + + \sum_{i=0}^{k-1}(c_9 + c_{10}) + c_{11} + \sum_{i=0}^{n} c_{12} \\
&\quad + \sum_{i=0}^{n-1}(c_{13} + c_{14} + c_{15}) + c_{16} \\
&= n(c_4 + c_5 + c_6 + c_{12} + c_{13} + c_{14} + c_{15}) \\
&\quad + k(c_8 + c_9 + c_{10}) + c_1 + c_2 + c_3 + c_4 \\
&\quad + c_7 + c_8 + c_{12} + c_{16} \\
&= n(c_4 + c_5 + c_6 + c_{12} + c_{13} + c_{14} + c_{15} + 1) \\
&\quad + k(c_8 + c_9 + c_{10} + 1) + c_3 + c_4 + c_7 + c_8 \\
&\quad + c_{12} + c_{16} + 4
\end{aligned}$$

Donc, la complexité temporelle est $\mathcal{O}(n + k)$.

Note : le dernier développement est si on développe les coûts c_1 et c_2 qui sont les seuls dont leur coût n'est pas constant à chaque fois qu'on effectue une instance de ces dernières.

b) La complexité spatiale de cet algorithme est $\mathcal{O}(n+k)$.

Exercice 5

a) L'algorithme transforme la matrice passée en paramètre en matrice triangulaire inférieure : pour chaque case de la partie triangulaire supérieure, il ajoute à la case correspondante de la partie triangulaire inférieure (de façon à peu près égale selon la ligne correspondante) la valeur déjà présente dans la case, plus la somme des éléments faisant partie de la partie triangulaire inférieure.

b) Une opération barométrique possible est l'instruction se trouvant à la ligne 7. En effet, la comparaison est effectuée $\sum_{i=0}^{n-1}\sum_{j=i+1}^{n} 1$ fois. La raison de ce choix est que, parmi toutes les instructions présentes dans la boucle principale entre les lignes 5 et 19, les instructions qui sont exécutées le plus souvent sont les instructions de comparaison des boucles aux lignes 7 et 14.

c)

$$\begin{aligned}
\tau(\text{WERK}) &= \sum_{i=0}^{n-1}\sum_{j=i+1}^{n} 1 \\
&= \sum_{i=0}^{n-1}(\sum_{j=0}^{n} 1 - \sum_{j=0}^{i} 1) \\
&= \sum_{i=0}^{n-1}((n+1)-(i+1)) \\
&= \sum_{i=0}^{n-1} n - \sum_{i=0}^{n-1} i \\
&= n \times n - \frac{n(n-1)}{2} \\
&= \frac{n^2-n}{2} \in \mathcal{O}(n^2)
\end{aligned}$$

Exercice 6

Approche naïve

a)

```
1   def somme_diagonale(M):
2       somme = 0
3       i = 0
4       while i < len(M):
5           j = 0
6           while j < len(M):
7               if i == j or j = (len(M) - i - 1):
8                   somme = somme + M[i][j]
9               j = j + 1
10          i = i + 1
11      return somme
```

b) Une opération barométrique possible dans cet algorithme est l'instruction se situant à la ligne 6. En effet, l'instruction de comparaison effectuée à l'intérieur de cette boucle imbriquée s'exécute exactement $\sum_{i=0}^{n-1}\sum_{j=0}^{n} 1$ fois. Comparée à certaines autres instructions, cette dernière est la seule qui s'exécute exactement $\sum_{i=0}^{n-1}\sum_{j=0}^{n} 1$ fois.

c)

$$\begin{aligned}
\tau(\text{SOMMEDIAGONALE}) &= \sum_{i=0}^{n-1}\sum_{j=0}^{n} 1 \\
&= \sum_{i=0}^{n-1}(n+1) \\
&= (n+1)\sum_{i=0}^{n-1} 1 \\
&= (n+1)n \\
&= n^2 + n \in \mathcal{O}(n^2)
\end{aligned}$$

Approche plus efficace

a)

```
1   def somme_diagonale(M):
2       somme = 0
3       milieu = 0
4       if n % 2 == 1:
5           milieu = n // 2
6       i = 0
7       while i < len(M):
8           somme = somme + M[i][i]
9           i = i + 1
10      j = i - 1
11      i = 0
12      while i < len(M):
13          somme = somme + M[i][j - i]
14          i = i + 1
15      if milieu > 0:
16          somme = somme - M[milieu][milieu]
17      return somme
```

b) Un choix possible pour l'opération barométrique dans cet algorithme est l'instruction de comparaison effectuée dans la boucle de la ligne 7. En effet, excepté l'instruction effectuée à la ligne 12, elle est la seule instruction qui est exécutée exactement n fois, où n représente les dimensions de la matrice M. Le reste des instructions sont exécutées en temps constant, ou bien $n-1$ fois pour les instructions aux lignes 8 et 13, qui auraient pu être aussi des choix possibles.

c)

$$\begin{aligned}\tau(\text{SOMMEDIAGONALE}) &= \sum_{i=0}^{n} 1 \\ &= n+1 \in \mathcal{O}(n)\end{aligned}$$

Série 2

Exercice 1

Pour démontrer que

$$f+g \in \Theta(\max(f,g))$$

il faut démontrer que $f+g \in \mathcal{O}(\max(f,g))$ et que $f+g \in \Omega(\max(f,g))$.

Démonstration de la borne supérieure

Démontrons que $f+g \in \mathcal{O}(\max(f,g))$. Pour pouvoir démontrer cette borne, il faut que la démonstration reste valide dans le cas où $\max(f(n),g(n)) = f(n)$ et dans le cas où $\max(f(n),g(n)) = g(n)$. Démontrons le premier cas.

$$\begin{aligned}f(n)+g(n) &\leq \max(f(n),g(n)) \\ &\leq f(n) && (\forall n, n \geq 0) \\ &\leq f(n)+f(n) && (\forall n, n \geq 0) \\ &\leq 2f(n) && (\forall n, n \geq 0)\end{aligned}$$

On trouve la paire suivante : $c = 2, n_0 = 0$ dans le cas où $\max(f(n), g(n)) = f(n)$. Démontrons le deuxième cas.

$$\begin{aligned} f(n) + g(n) &\leq \max(f(n), g(n)) \\ &\leq g(n) && (\forall n, n \geq 0) \\ &\leq g(n) + g(n) && (\forall n, n \geq 0) \\ &\leq 2g(n) && (\forall n, n \geq 0) \end{aligned}$$

La paire $c = 2, n_0 = 0$ valide le deuxième cas.

Donc, $f + g \in \mathcal{O}(\max(f, g))$.

Démonstration de la borne inférieure

Démontrons que $f + g \in \Omega(\max(f, g))$.

$$f(n) + g(n) \geq \max(f(n), g(n)) \qquad (\forall n, n \geq 0)$$

La paire $c = 1, n_0 = 0$ confirme que $f + g \in \Omega(\max(f, g))$.

En conclusion, $f + g \in \Theta(\max(f, g))$.

Exercice 2

Démonstration de la borne supérieure

$$\begin{aligned} (n + a)^b &\leq n^b && (\forall n, n \geq 0) \\ &\leq (n + n)^b && (\forall n, n \geq a) \\ &\leq (2n)^b && (\forall n, n \geq a) \\ &\leq 2^b n^b && (\forall n, n \geq a) \end{aligned}$$

La paire $c = 2^b$ et $n_0 = a$ valide que $(n + a)^b \in \mathcal{O}(n^b)$.

Démonstration de la borne inférieure

$$(n + a)^b \geq n^b \qquad (\forall n, n \geq 0)$$

La paire $c = 1$ et $n_0 = 0$ valide que $(n + a)^b \in \Omega(n^b)$.

Donc, $(n + a)^b \in \Theta(n^b)$.

Exercice 3

Note : afin d'éviter toute confusion, α représente la première constante dans le cas de la borne supérieure et β celle de la borne inférieure.

Démonstration de la borne supérieure

$$\begin{aligned}
\sum_{i=1}^{n} i^c &\leq \sum_{i=1}^{n} i^c && (\forall n, n \geq 0) \\
&\leq \sum_{i=1}^{n} n^c && (\forall n, n \geq 0) \\
&\leq n^c \times n && (\forall n, n \geq 0) \\
&\leq n^{c+1} && (\forall n, n \geq 0)
\end{aligned}$$

La paire $\alpha = 1$ et $n_0 = 0$ valide que $\sum_{i=1}^{n} i^c \in \mathcal{O}(n^{c+1})$.

Démonstration de la borne inférieure

$$\begin{aligned}
\sum_{i=1}^{n} i^c &\geq \sum_{i=1}^{n} i^c && (\forall n, n \geq 0) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \frac{n^c}{2^c} && (\forall n, n \geq 0) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \left(\frac{n}{2}\right)^c && (\forall n, n \geq 0) \\
&\geq \left(\frac{n}{2}\right)^c \times \left\lfloor \frac{n}{2} \right\rfloor && (\forall n, n \geq 0) \\
&\geq \frac{1}{2^{c+1}} n^{c+1} && (\forall n, n \geq 0)
\end{aligned}$$

La paire $\beta = \frac{1}{2^{c+1}}$ et $n_0 = 0$ valide que $\sum_{i=1}^{n} i^c \in \Omega(n^{c+1})$.

En conclusion, $\sum_{i=1}^{n} i^c \in \Theta(n^{c+1}), c > -1$.

Exercice 4

Voici l'ordre des fonctions, par ordre de grandeur croissant :

$$\log^2 n, 2^{\log n}, \log n!, n^3, (\log n)^{\log n}, \left(\sqrt{n}\right)^{\log n}, \left(\frac{3}{2}\right)^n, 2^{n+1}, 2^{2n}, n!, 2^{2^n}$$

Exercice 5

a) Vrai, la constante multiplicative dans $2n$ ne change pas la complexité asymptotique dans ce contexte et donc, $2n \in \Theta(n)$.

b) Faux, avec la notation o, une fonction ne peut pas croître strictement plus vite qu'elle-même.
c) Faux, avec la notation ω, une fonction ne peut pas croître strictement plus vite qu'elle-même.
d) Faux, si on tente de démontrer que $n! \in \Omega((n+1)!)$, on tombe sur la contradiction suivante :

$$\begin{aligned} n! &\geq c(n+1)! \\ n! &\geq c(n+1)n! \\ 1 &\geq c(n+1) \qquad \text{Contradiction} \end{aligned}$$

Il n'existe pas de constantes c et n_0 telles que $n! \in \Omega((n+1)!)$. Donc, $n! \notin \Theta((n+1)!)$.
e) Vrai, la sommation de gauche se simplifie en n^2. Or, dans le cas Ω, la fonction peut croître plus vite ou à la même vitesse qu'elle-même.
f) Faux, dans le cas de fonctions exponentielles qui n'ont pas la même base, c'est celle qui a la plus grande base qui domine et donc $2^n \notin \Omega(3^n)$.
g) Faux, en simplifiant la sommation, le terme dominant est n^2, qui croît plus vite que n. Donc, $n^2 \notin \Theta(n)$.
h) Vrai, deux logarithmes de bases différentes croissent à la même vitesse. En effet, on peut changer la base d'un logarithme, ce qui ajoute simplement un facteur constant, sans effet dans ce contexte.

Exercice 6

a) Si $f(n) = 2n$ et $g(n) = n$, alors $f \in \mathcal{O}(g)$, mais $2^f \notin \mathcal{O}(2^g)$.
b) Si $f(n) = n^2$ et $g(n) = n$, alors $f + g \notin \Theta(\min(f, g))$, car $f(n)$ est le terme dominant, alors que $\min(f, g) = g$ ne croît pas à la même vitesse. En effet, $f \in \omega(g)$.
c) Si $f(n) = n^2$ et $g(n) = n$, alors $f \in \Omega(g)$, mais $g \notin \Omega(f)$, car $g \in o(f)$. En effet, la fonction g croît strictement moins vite que la fonction f.

Annexe

Sommation

Sommation 1

$$
\begin{aligned}
\sum_{i=k}^{m}\sum_{a=j}^{n} i &= \sum_{i=k}^{m} i \sum_{a=j}^{n} 1 \\
&= \sum_{i=k}^{m} i(\sum_{a=0}^{n} 1 - \sum_{a=0}^{j-1} 1) \\
&= \sum_{i=k}^{m} i(n+1-j) \\
&= (n+1-j)\sum_{i=k}^{m} i \\
&= (n+1-j)\left(\sum_{i=0}^{m} i - \sum_{i=0}^{k-1} i\right) \\
&= (n+1-j)\left(\frac{m(m+1)}{2} - \frac{k(k-1)}{2}\right) \\
&= (n+1-j)\left(\frac{m^2+m}{2} - \frac{k^2-k}{2}\right) \\
&= (n+1-j)\left(\frac{m^2+m-k^2+k}{2}\right) \\
&= (n+1-j)\left(\frac{(m+k)(m-k+1)}{2}\right) \\
&= \frac{(n+1-j)(m+k)(m-k+1)}{2}
\end{aligned}
$$

Sommation 2

$$\begin{aligned}\sum_{i=1}^{n}(2i-1) &= \sum_{i=1}^{n}2i - \sum_{i=1}^{n}1 \\ &= 2\sum_{i=1}^{n}i - \sum_{i=1}^{n}1 \\ &= 2\left(\frac{n(n+1)}{2}\right) - n \\ &= n^2 + n - n \\ &= n^2\end{aligned}$$

Sommation 3

$$\begin{aligned}\sum_{j=0}^{n-1}j2^j &= \sum_{j=0}^{n}(j2^j) - n2^n \\ &= \frac{n2^{n+2} - (n+1)2^{n+1} + 2}{(1-2)^2} - n2^n \\ &= n2^{n+2} - (n+1)2^{n+1} + 2 - n2^n \\ &= n2^{n+2} - n2^{n+1} - 2^{n+1} + 2 - n2^n \\ &= 2 + 2^n(4n - 2n - 2 - n) \\ &= (n-2)2^n + 2\end{aligned}$$

Sommation 4

$$\begin{aligned}
\sum_{i=0}^{\beta}(i^2) - \sum_{j=0}^{\beta}(\beta - j) &= \sum_{i=0}^{\beta}(i^2) - (\sum_{j=0}^{\beta}\beta - \sum_{j=0}^{\beta} j) \\
&= \left(\frac{\beta(\beta+1)}{2}\right)^2 - \left(\beta(\beta+1) - \frac{\beta(\beta+1)}{2}\right) \\
&= \left(\frac{\beta(\beta+1)}{2}\right)^2 - \frac{\beta(\beta+1)}{2} \\
&= \frac{\beta(\beta+1)}{2}\left(\frac{\beta(\beta+1)}{2} - 1\right) \\
&= \frac{\beta(\beta+1)}{2}\left(\frac{\beta^2+\beta-2}{2}\right) \\
&= \frac{\beta(\beta+1)(\beta+2)(\beta-1)}{4}
\end{aligned}$$

Sommation 5

$$\sum_{d=n+1}^{n} d^4 = 0$$

Sommation 6

$$\begin{aligned}
\sum_{\alpha=1}^{k}(\frac{3}{2})^{\alpha} &= \frac{1-(\frac{3}{2})^{k+1}}{1-\frac{3}{2}} \\
&= \frac{1-(\frac{3}{2})^{k+1}}{-\frac{1}{2}} \\
&= 3\left(\frac{3}{2}\right)^k - 2
\end{aligned}$$

Sommation 7

$$\begin{aligned}\sum_{i=0}^{\log_3 n}(\frac{2}{3})^i &= \frac{1-(\frac{2}{3})^{\log_3 n+1}}{1-\frac{2}{3}}\\ &= \frac{1-(\frac{2}{3})^{\log_3 n+1}}{\frac{1}{3}}\\ &= 3-2\left(\frac{2}{3}\right)^{\log_3 n}\end{aligned}$$

Sommation 8

$$\begin{aligned}\sum_{i=0}^{\log_3 n} n(\frac{2}{3})^i &= n(\frac{1-(\frac{2}{3})^{\log_3 n+1}}{1-\frac{2}{3}})\\ &= n(\frac{1-(\frac{2}{3})^{\log_3 n+1}}{\frac{1}{3}})\\ &= \frac{n}{3}-2n\left(\frac{2}{3}\right)^{\log_3 n}\\ &= \frac{n}{3}-2n(n^{\log_3 \frac{2}{3}})\\ &= \frac{n}{3}-2n^{\log_3 2}\end{aligned}$$

Sommation 9

$$\begin{aligned}\sum_{k=2}^{p} k &= \sum_{k=0}^{p} k - \sum_{k=0}^{1} k\\ &= \frac{k(k+1)}{2}-1\\ &= \frac{k^2+k-2}{2}\\ &= \frac{(k+2)(k-1)}{2}\end{aligned}$$

Sommation 10

$$\begin{aligned}
\sum_{p=0}^{n}(\sum_{p=0}^{n}5+\sum_{p=0}^{n}2) &= \sum_{p=0}^{n}(5(n+1)+2(n+1)) \\
&= \sum_{p=0}^{n}(7n+7) \\
&= \sum_{p=0}^{n}7n+\sum_{p=0}^{n}7 \\
&= 7n(n+1)+7(n+1) \\
&= (n+1)(7n+7) \\
&= 7(n+1)^2
\end{aligned}$$

Sommation 11

$$\begin{aligned}
\sum_{\theta=0}^{n}(\theta-\sum_{\psi=0}^{m}2\psi) &= \sum_{\theta=0}^{n}(\theta-2(m+1)) \\
&= \sum_{\theta=0}^{n}\theta-\sum_{\theta=0}^{n}(2m+2) \\
&= \frac{n(n+1)}{2}-(2m(n+1)+2n+2) \\
&= \frac{n^2+n-4mn-2m-2n-2}{2} \\
&= \frac{n^2-n-4mn-2m-2}{2}
\end{aligned}$$

Sommation 12

$$\begin{aligned}
\sum_{i=1}^{\log n} \log(\frac{n}{2^i}) &= \sum_{i=1}^{\log n} \log n - \sum_{i=1}^{\log n} \log 2^i \\
&= \eta \sum_{i=1}^{\eta} 1 - \sum_{i=1}^{\eta} i && \text{Posons } \eta = \log n \\
&= \eta^2 - \frac{\eta(\eta+1)}{2} \\
&= \frac{\eta^2 - \eta}{2} \\
&= \frac{\log^2 n - \log n}{2}
\end{aligned}$$

Sommation 13

$$\begin{aligned}
\sum_{s=0}^{n-1} s &= \sum_{s=0}^{n} s - s \\
&= \frac{s(s+1)}{2} - s \\
&= \frac{s^2 - s}{2} \\
&= \frac{s(s-1)}{2}
\end{aligned}$$

Sommation 14

$$\begin{aligned}
\sum_{k=2\sigma}^{m} k &= \sum_{k=0}^{m} k - \sum_{k=0}^{2\sigma-1} k \\
&= \frac{m(m+1)}{2} - \frac{2\sigma(2\sigma-1)}{2} \\
&= \frac{m^2 + m - 4\sigma^2 + 2\sigma}{2} \\
&= \frac{(m+2\sigma)(m-2\sigma+1)}{2}
\end{aligned}$$